Introduction to Artificial Intelligence

Semester I, 2026-27

Problem Solving as Search

Rohan Paul

This Class

- Reflex Agents
- Problem Solving as search
  - Uninformed Search
- Reference Material
  - AIMA Ch. 3

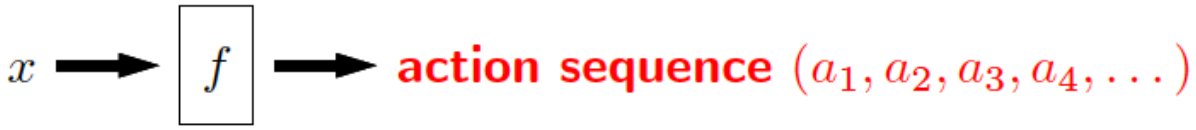
Acknowledgement

These slides are intended for teaching purposes only. Some material has been used/adapted from web sources and from slides by Doina Precup, Dorsa Sadigh, Percy Liang, Mausam, Dan Klein, Nicholas Roy and others.

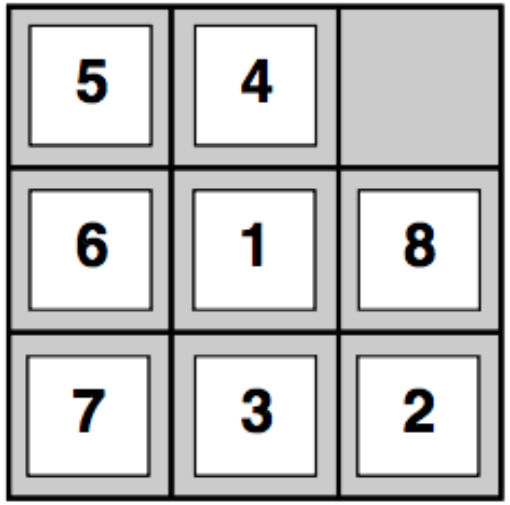
Thinking of Problem Solving in a General Way

Problem Solving Agents

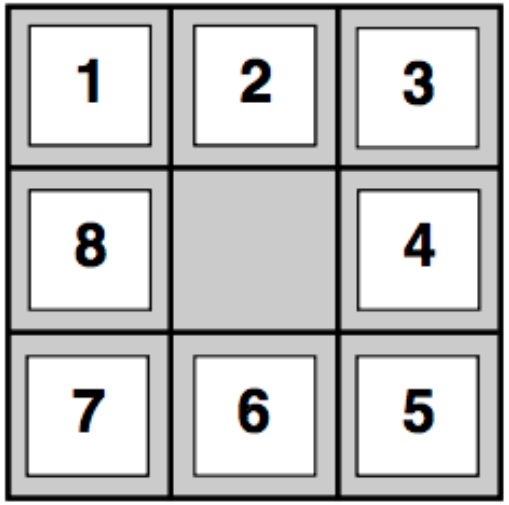
- Problem Solving Agents
  - Adopt a goal
  - Consider future actions and the desirability of their outcomes
  - Solution: a sequence of actions that the agent can execute leading to the goal.
  - Today's focus.



Example – The Eight Puzzle



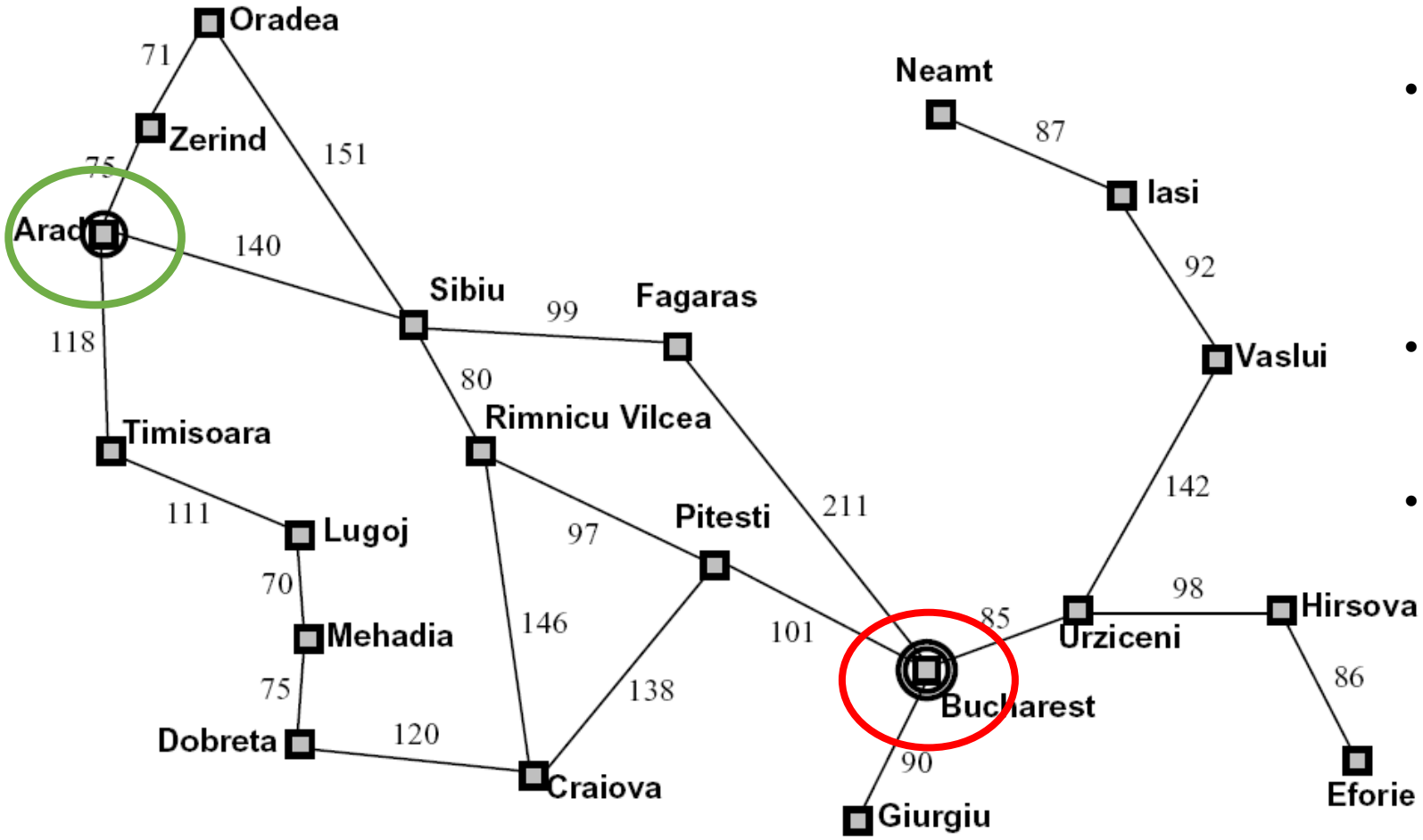
Start State



Goal State

- States: configurations of the puzzle
- Goals: target configuration
- Operators: swap the blank with an adjacent tile
- Path cost: number of moves

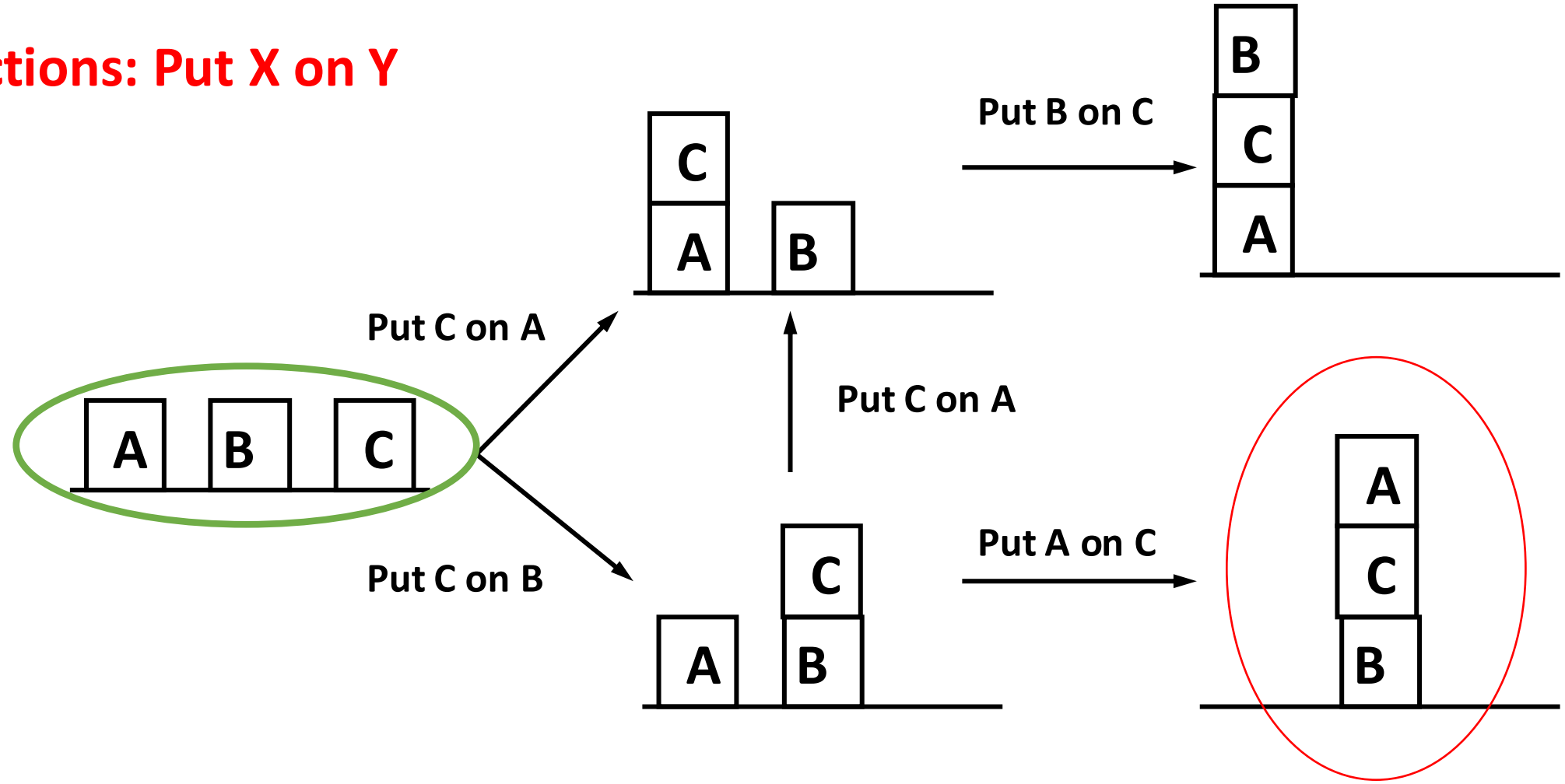
Example – Route Finding



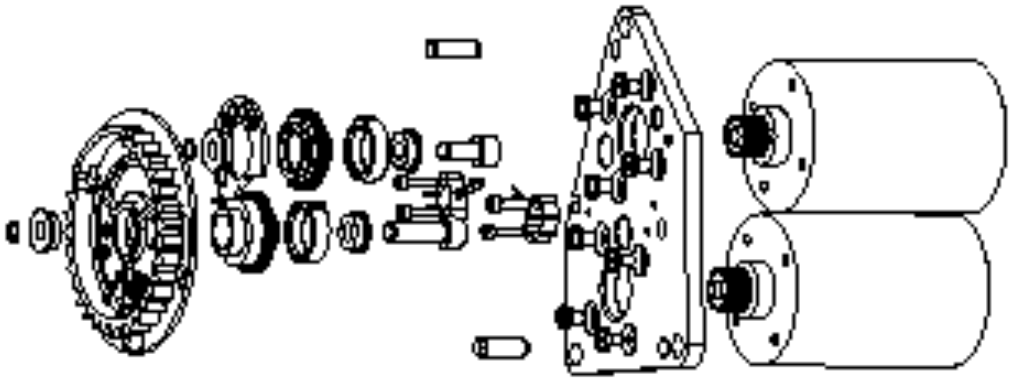
- Problem:
  - Find a solution i.e., a sequence of actions (road traversals) that can take the agent to the destination in minimum time.
- Search
  - Process of looking for a sequence of actions that reaches the goal
- Note: as we will see search problems are more general than path finding.

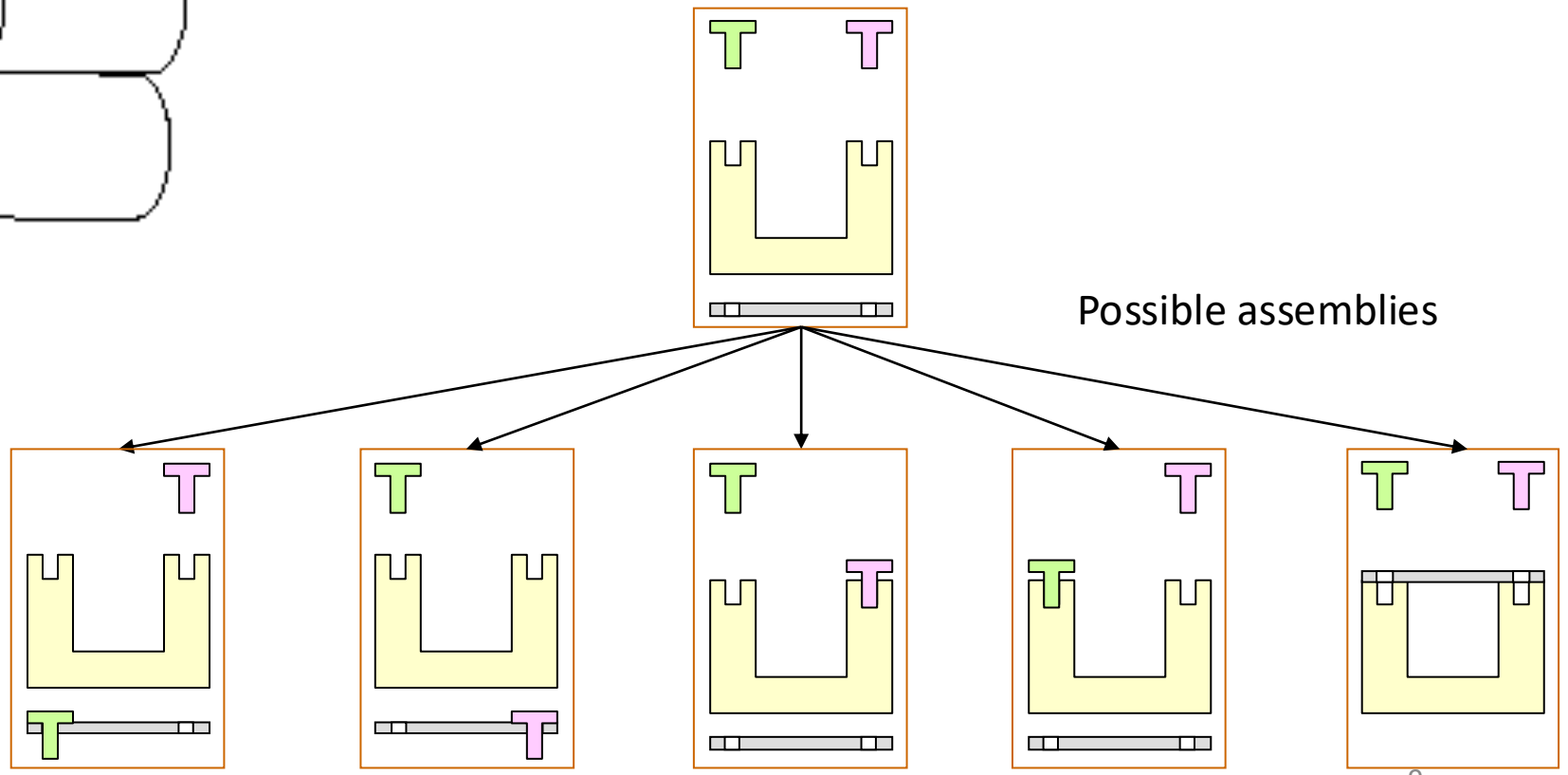
Example – Block Manipulation

Actions: Put X on Y

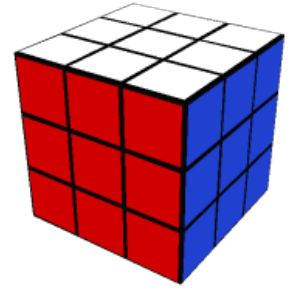


Examples – Assembly Planning

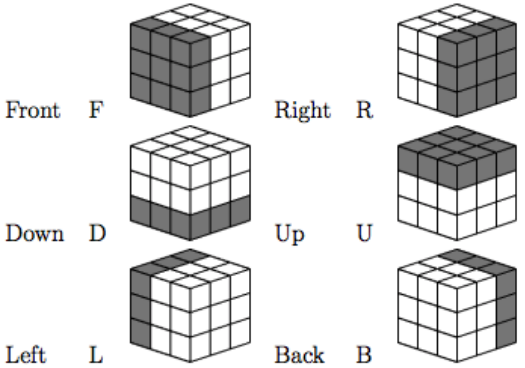




Example: Rubric's Cube



- Setup
  - Single goal state (correction)
  - Large number of possible states (4.3 x 10^19 states).
  - Randomly exploring is unlikely to give a solution.
  - Different ways to represent actions.
    - https://solvethecube.com/notation
  - Modern attempts at solving (advanced techniques)
    - https://www.nature.com/articles/s42256-019-0070-z
    - Combination of classical search and learning based approaches.
  - There is fundamental symmetries and structure



Search Problem Formulation

Many problems in AI can be modeled as search problems.

- *State space* S: all possible configurations of the domain of interest
- *An initial (start) state* $s_0 \in S$
- *Goal states* $G \subset S$: the set of end states
  - Often defined by a *goal test* rather than enumerating a set of states
- *Operators* A: the actions available
  - Often defined in terms of a *mapping from a state to its successor*

When an action is executed, the state changes.

What we want to achieve?

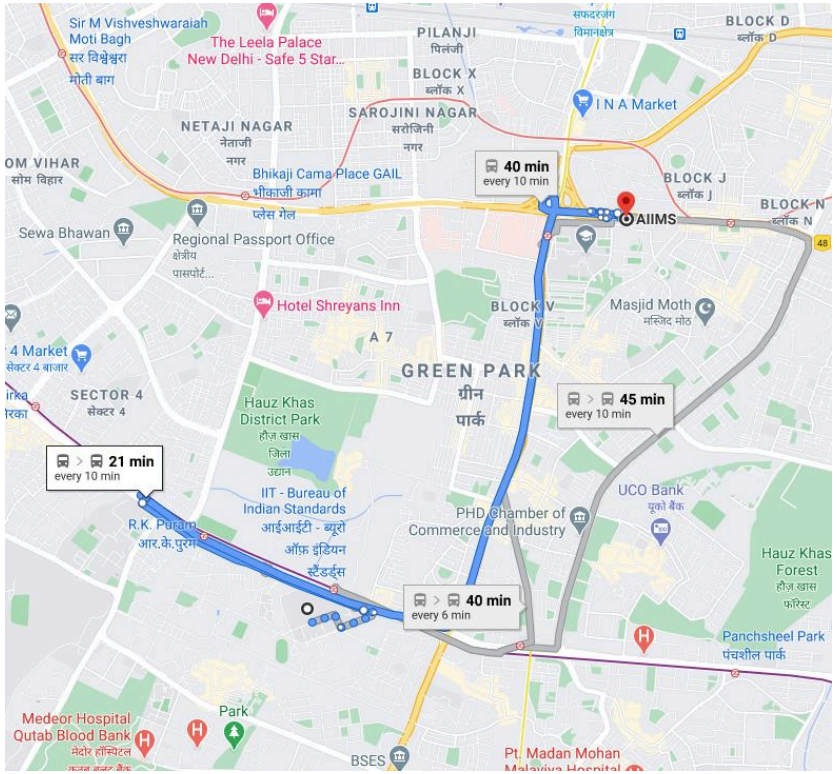
A sequence of actions that take us from the start to the goal state.

The sequence of actions optimizes some objective (desirability).

Formulating a Search Problem

- *Path*: a sequence of states and operators
- *Path cost*: a number associated with any path
  - Measures the quality of the path
  - Usually the smaller, the better

- Find a solution which is a sequence of actions that transforms the start state to a goal state.
- Search is the process of looking for a sequence of actions that reaches the goal.

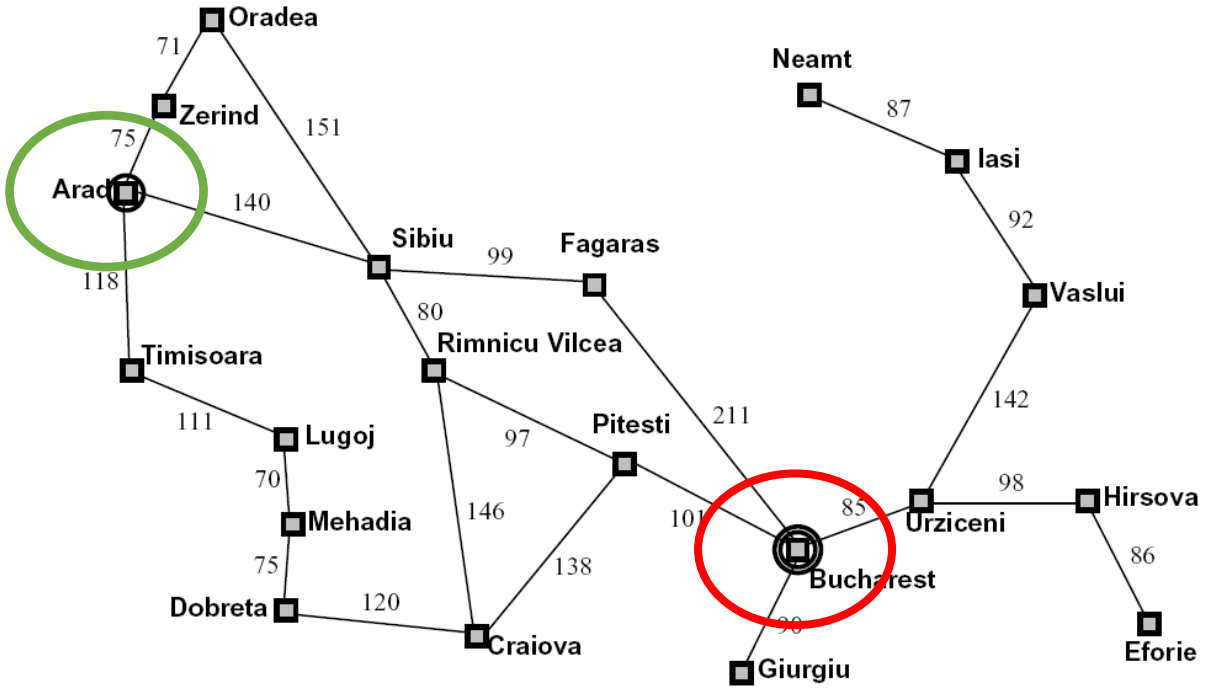


Route finding in a map.

Modeling assumptions for now

- Environment is observable
  - The agent always knows it current state.
- Discrete states and actions
  - Finite number of cities.
  - At any given state there are a finite number of actions.
- Known and Deterministic action outcomes
  - The agent knows which sates are reached by each action.
  - Action has exactly one outcome when applied to a state.

Example – Route Finding

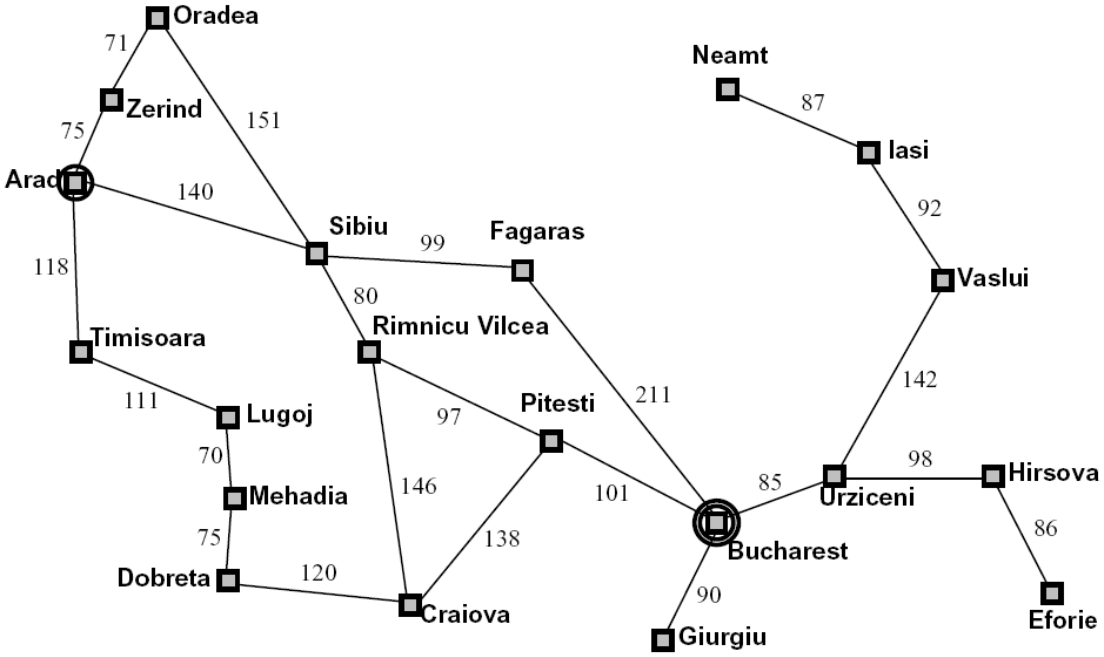


- State space:
  - All the cities on the map.
- Actions:
  - Traversing a road: Going to an adjacent city.
- Cost:
  - Distance along the road
- Start state:
  - Arad
- Goal test:
  - Is state == Bucharest?

Modeling Notions – State Space Graphs and Search Trees

Where all the agent can be? *State Space Graphs*

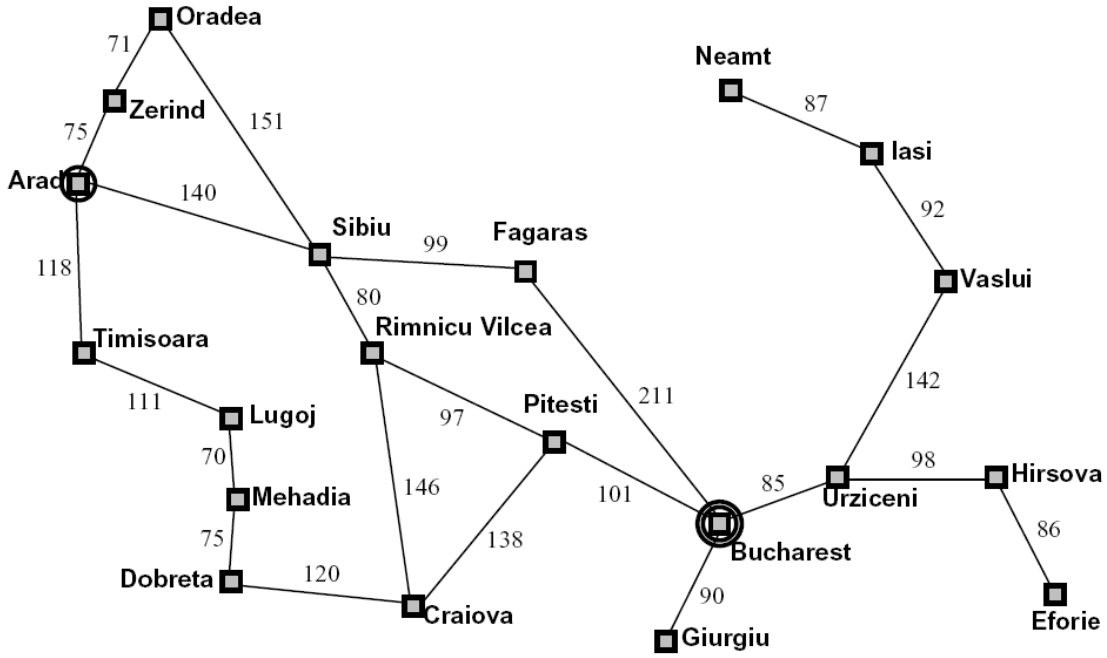
- A representation of a search problem
  - Nodes are (abstracted) world configurations
  - Arcs represent successors (action results)
  - The goal test is a set of goal nodes (maybe only one)
  - **Each state occurs only once**
- The full graph characterizes the space in which the solution is to be found.
  - It is usually too large.



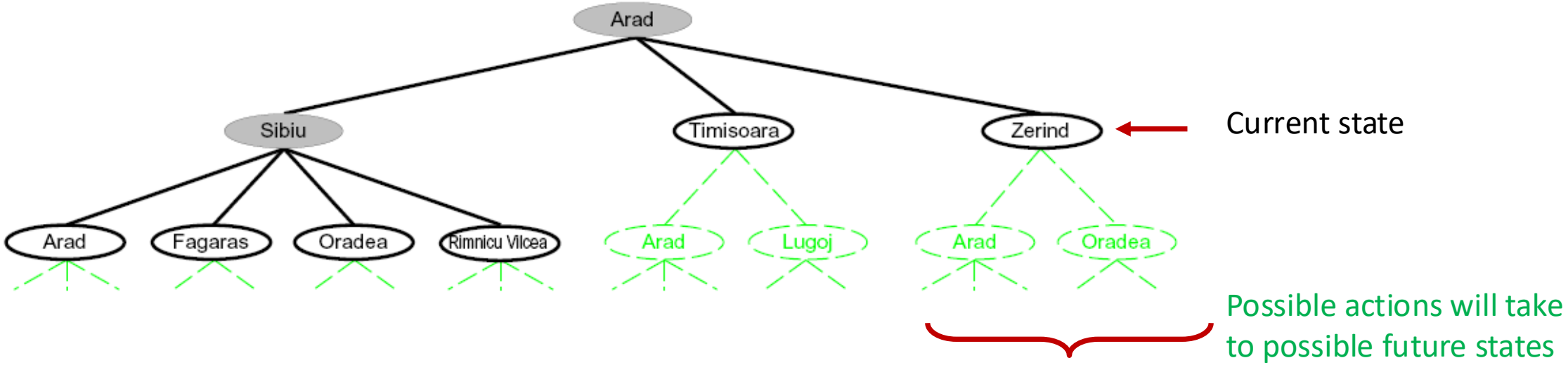
Need some AI technique that explore this graph implicitly by visualizing which actions are permitted and which states they can lead to. We do this for some steps in the future.

Searching for a solution

- Once the problem is formulated, need to solve it.
- Solution – action sequences. Search algorithms work by considering various possible action sequences.



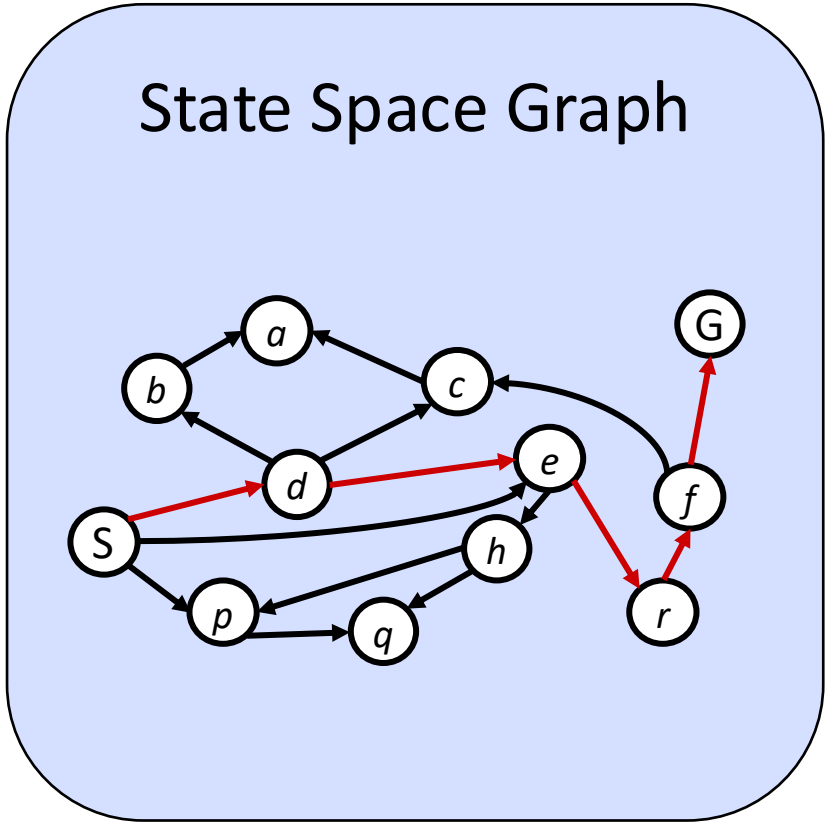
Search Trees

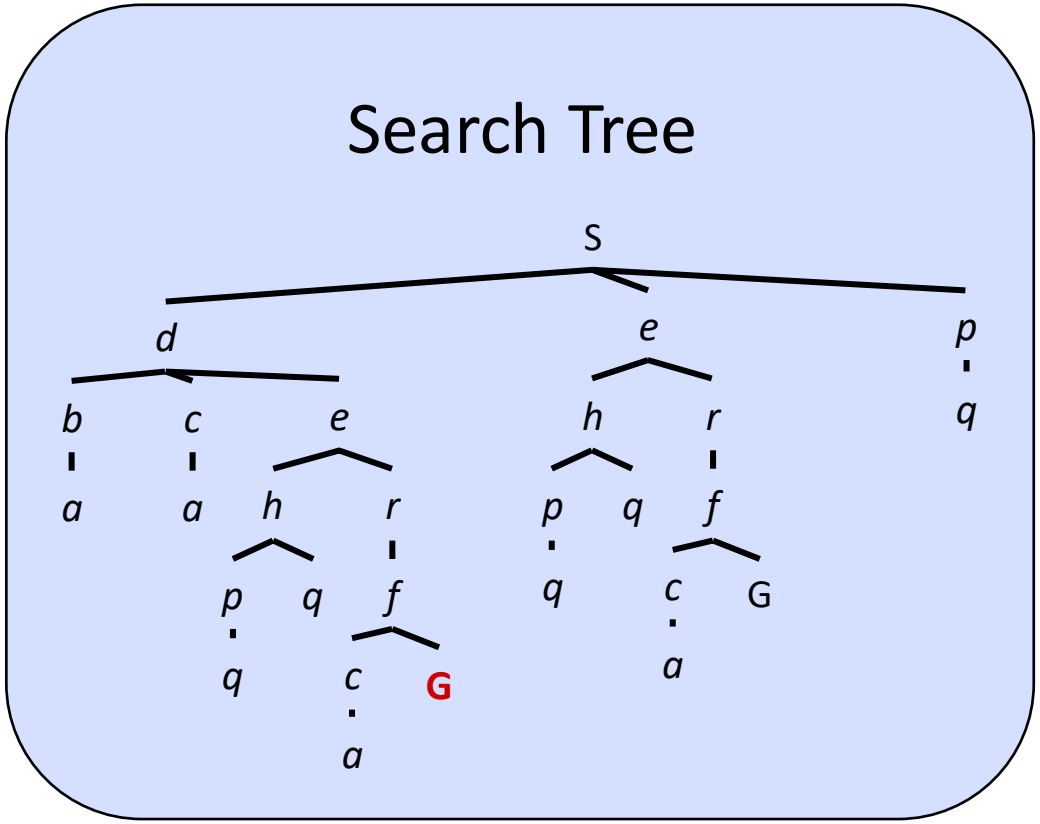


- A search tree: A "what if" tree of plans and their outcomes
  - The start state is the root node
  - Check if the node contains the goal.
  - Other wise, "expand" the node
    - Apply legal actions on the current state to generate new set of states.
  - Frontier
    - All the nodes available for expansion.
  - In a Search Tree, nodes show states, but correspond to PLANS that achieve those states

State Space Graph vs. Search Tree

- Each NODE in in the search tree is an entire PATH in the state space graph.
- Construct both on demand and construct as little as possible.



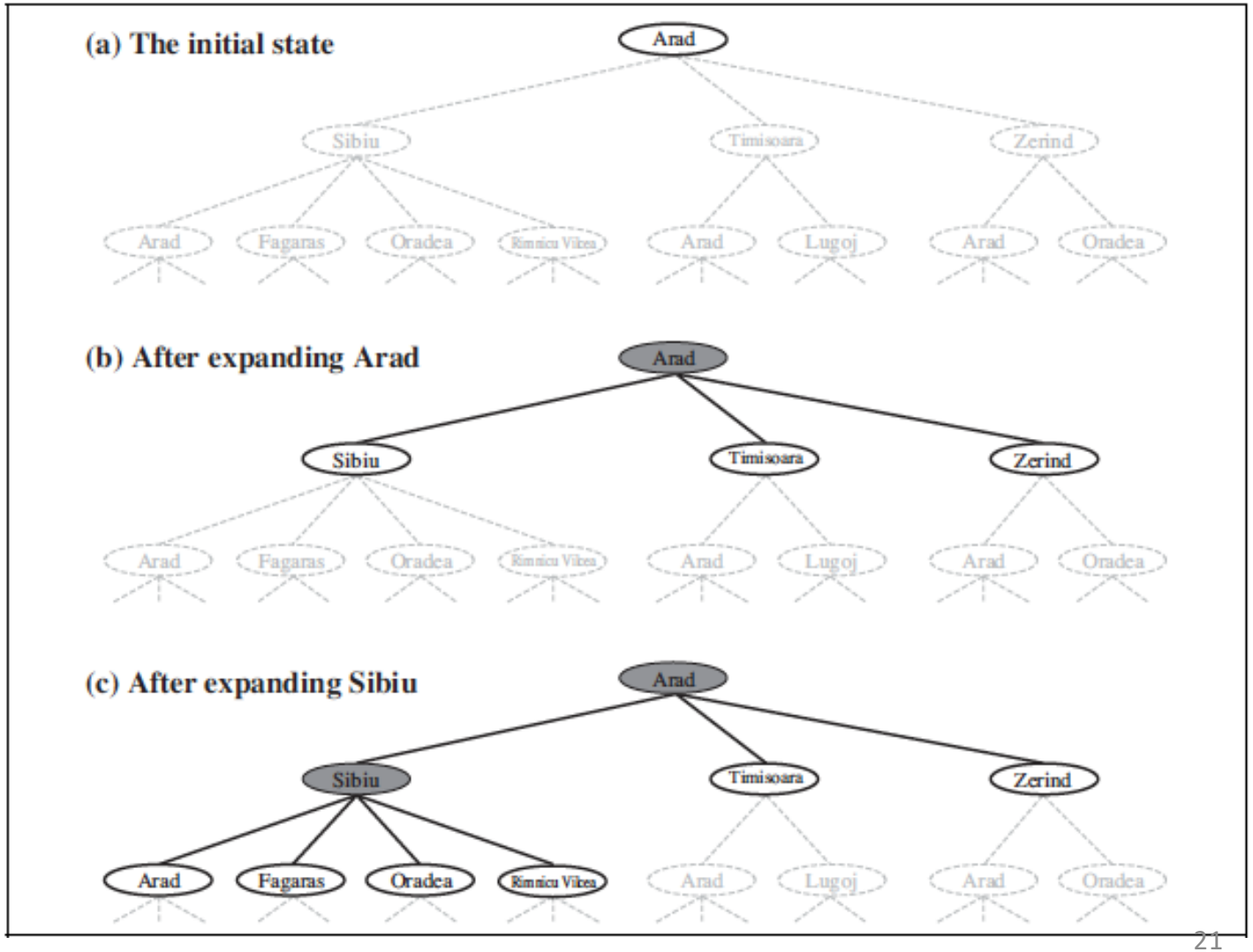


Tree Search

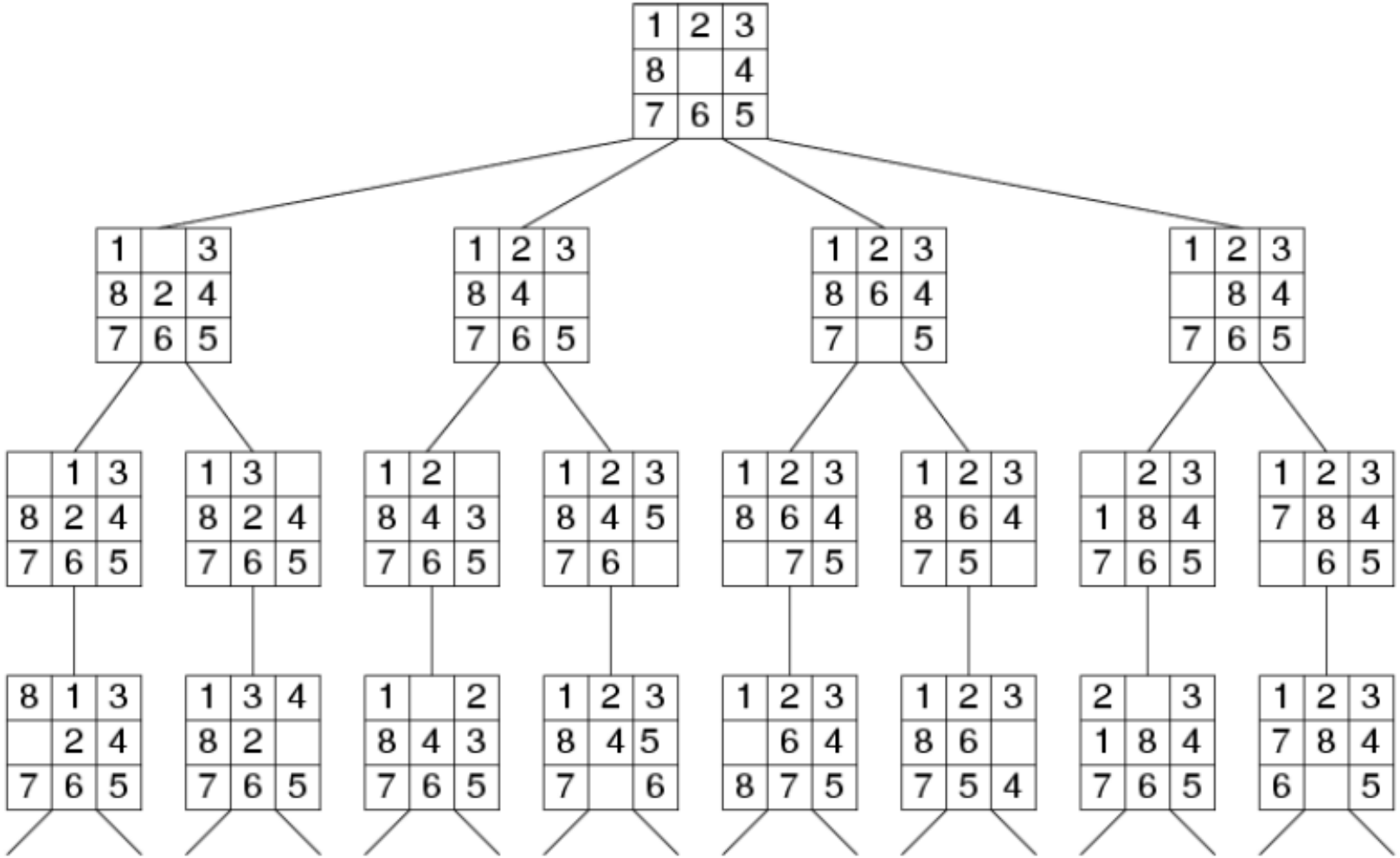
```
function TREE-SEARCH( problem, strategy) returns a solution, or failure
    initialize the search tree using the initial state of problem
    loop do
        if there are no candidates for expansion then return failure
        choose a leaf node for expansion according to strategy
        if the node contains a goal state then return the corresponding solution
        else expand the node and add the resulting nodes to the search tree
    end
```

Search Tree

- The process of expansion while constructing the search tree.
  - Note that Arad appears again.
- Loops lead to redundancy
  - Why? Path costs are additive.
- Can we remember which nodes were expanded?



The Eight Puzzle – State Space (Fragment)

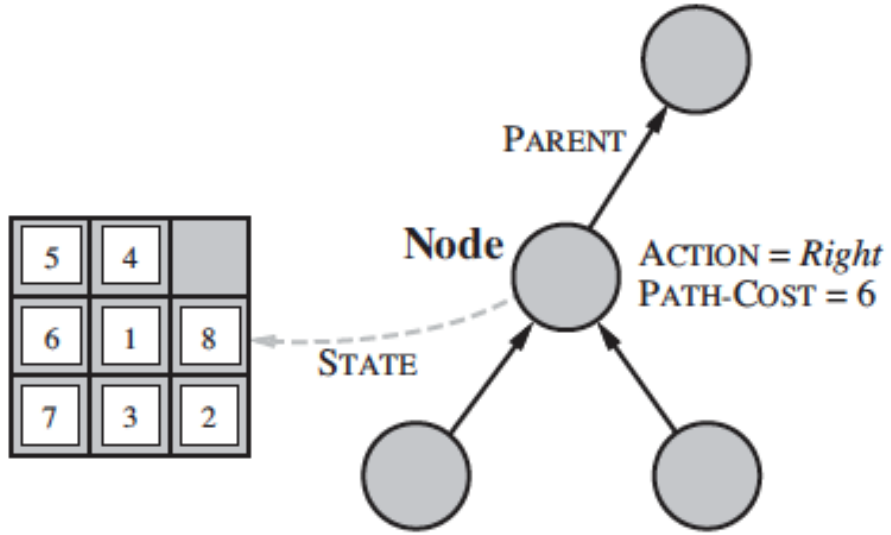


Search as an organized procedure to find solutions

Frontier as a way to organize partial solutions

Infrastructure for Search Algorithms

- *Defining a search node*:
  - Each node contains a state
  - Node also contains additional information, e.g.:
    - The parent state and the operator used to generate it
    - Cost of the path so far
    - Depth of the node
- *Expanding a node*:
  - Applying all legal operators to the state contained in the node
  - Generating nodes for all the corresponding successor states.



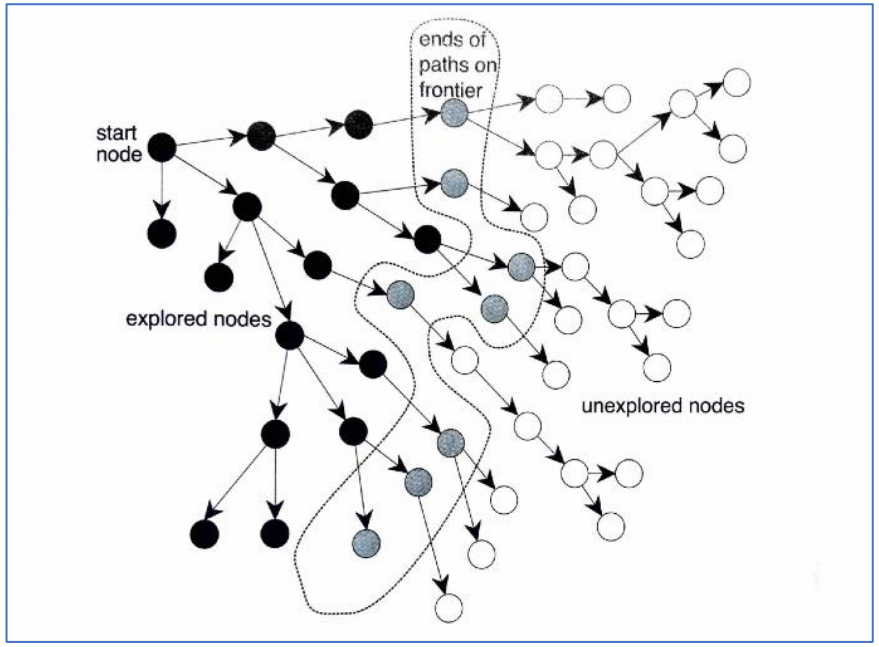

```
function CHILD-NODE(problem, parent, action) returns a node
  return a node with
      STATE = problem.RESULT(parent.STATE, action),
      PARENT = parent, ACTION = action,
      PATH-COST = parent.PATH-COST + problem.STEP-COST(parent.STATE, action)
```

Notion of a frontier

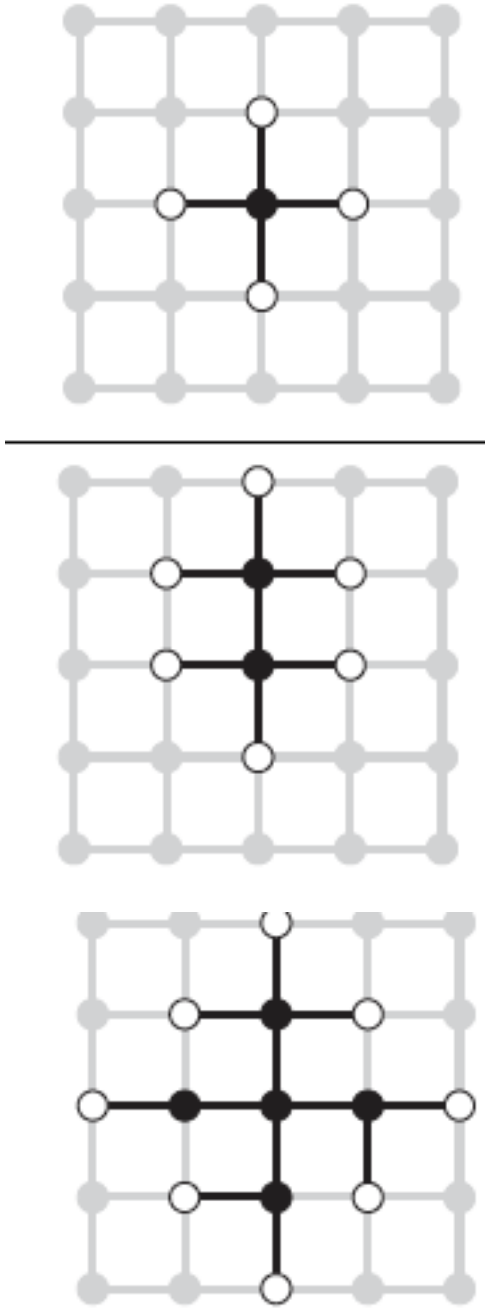
Note: we are doing problem solving as a ”search” over the state space graph. The graph is implicit and is considered as we go along.

How to manage generated nodes – Frontier (also called open list)?

- Need a data structure for managing nodes as they are generated.
- Queue (characterized by the order win which they store the inserted nodes).



Progression of search



The frontier represents partial solutions

- Our problem is to find an optimal sequence of actions from
  - The start state to the
  - Goal state

- Search visualizes actions and where do they lead to
  - They will expand what we have explored so far.
  - See if the solution is there. Otherwise expand further.

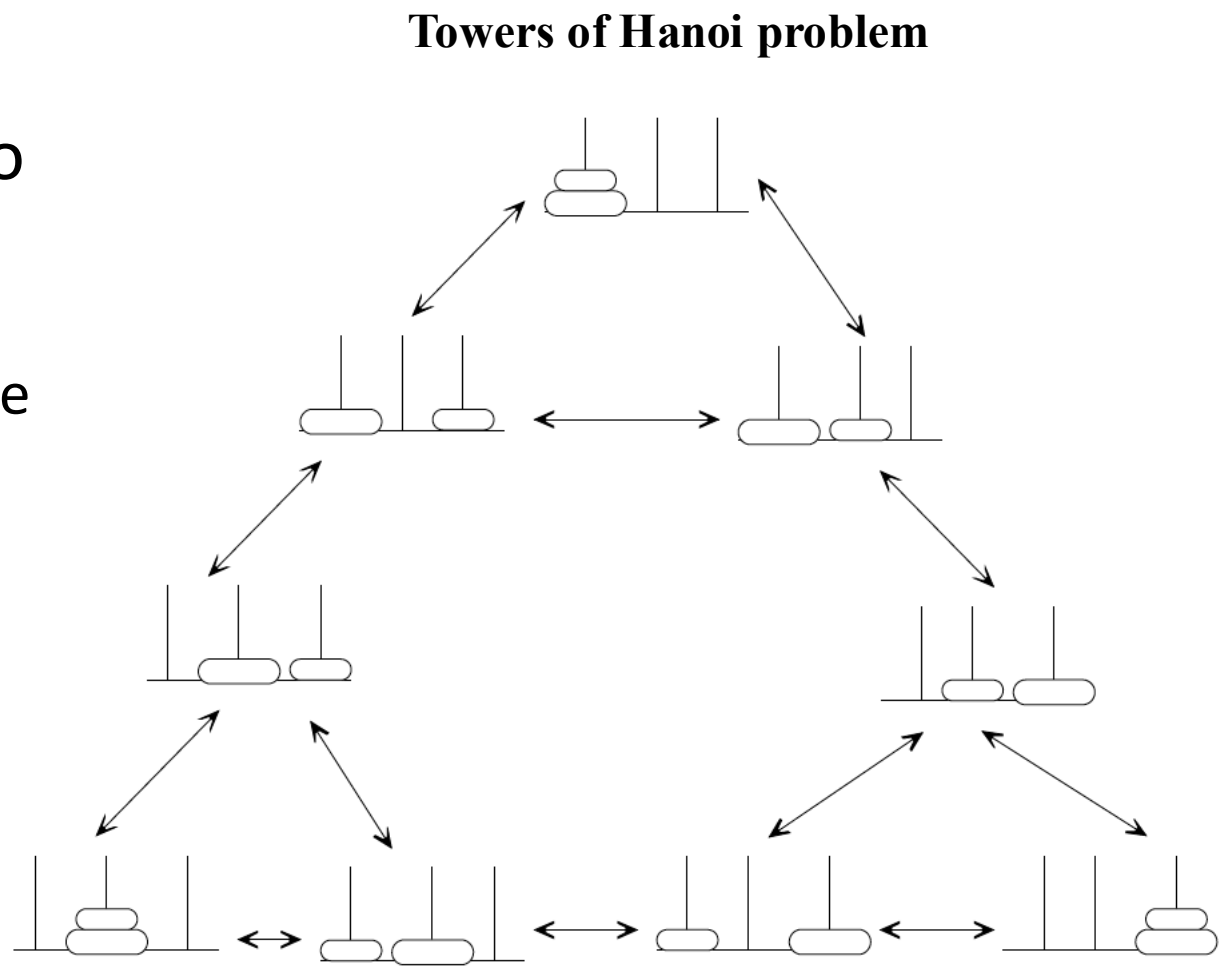
```
1:  procedure Search(G, S, goal)
2:      Inputs
3:          G: graph with nodes N and arcs A
4:          s: start node
5:          goal: Boolean function of nodes
6:      Output
7:          path from s to a node for which goal is true
8:          or ⊥ if there are no solution paths
9:      Local
10:         frontier: set of paths
11:     frontier := {⟨s⟩}
12:     while frontier ≠ {} do
13:         select and remove ⟨n_0, ..., n_k⟩ from frontier
14:         if goal(n_k) then
15:             return ⟨n_0, ..., n_k⟩
16:         frontier := frontier ∪ {⟨n_0, ..., n_k, n⟩ : ⟨n_k, n⟩ ∈ A}
17:     return ⊥
```

Problem of Repeated States

The need for “remembering” explored nodes

Repeated States Examples

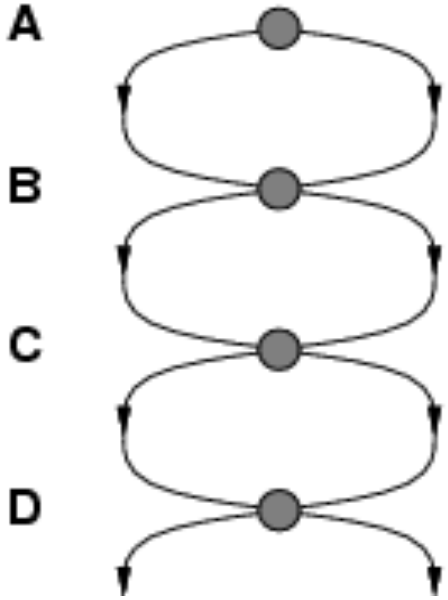
- Reversible actions can lead to repeated states.
  - Reversible actions, e.g., the 8 puzzle, tower of hanoi or route finding.
- Lead to loopy or redundant* paths in the tree search.



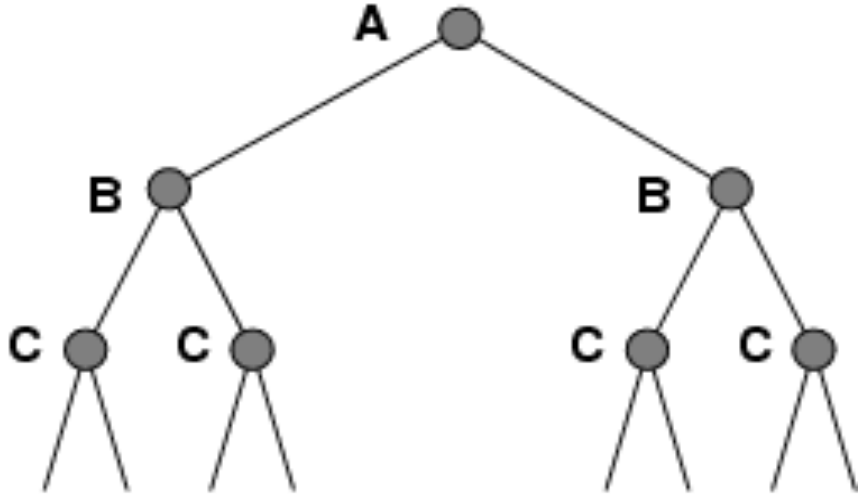
* Each additional path found is going to be longer.

Revisited states can significant increase search complexity

Original graph



Search tree without checking revisited states.



If we did not check for duplicate states, then the tree size is exponential in the number of states. If we do check for repeated states, then our tree is much smaller (linear).

Graph Search

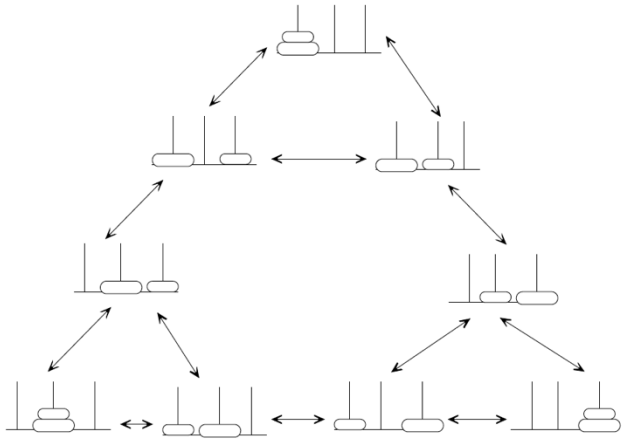
```
function GRAPH-SEARCH(problem) returns a solution, or failure
    initialize the frontier using the initial state of problem
    initialize the explored set to be empty
    loop do
        if the frontier is empty then return failure
        choose a leaf node and remove it from the frontier
        if the node contains a goal state then return the corresponding solution
        add the node to the explored set
        expand the chosen node, adding the resulting nodes to the frontier
            only if not in the frontier or explored set
```

Newly generated nodes that match the frontier nodes or expanded nodes are discarded.

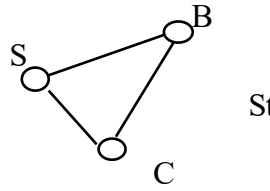
Repeated States Burden Search Algorithms

- What causes states to appear again during search?
  - Reversible actions can lead to repeated states.
    - Reversible actions, e.g., the 8 puzzle, tower of hanoi or route finding.
  - Genuine multiple paths to the goal.
- Lead to more than one way to arrive at a node. Adds additional plans to explore within the search process.
- However, they are longer and hence redundant hence not useful.
- How to not consider redundant paths in the tree search.
  - Remember all the nodes that have been explored. In addition to these nodes, also check nodes on the frontier before adding a new solution path for consideration.
- Let's think: What else can be done to either not generate solution candidates that we know will be longer or in general make the process of checking for loops faster?

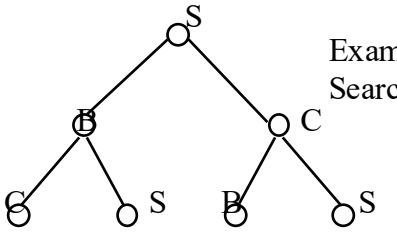
Towers of Hanoi problem



Route traversal problem



State Space

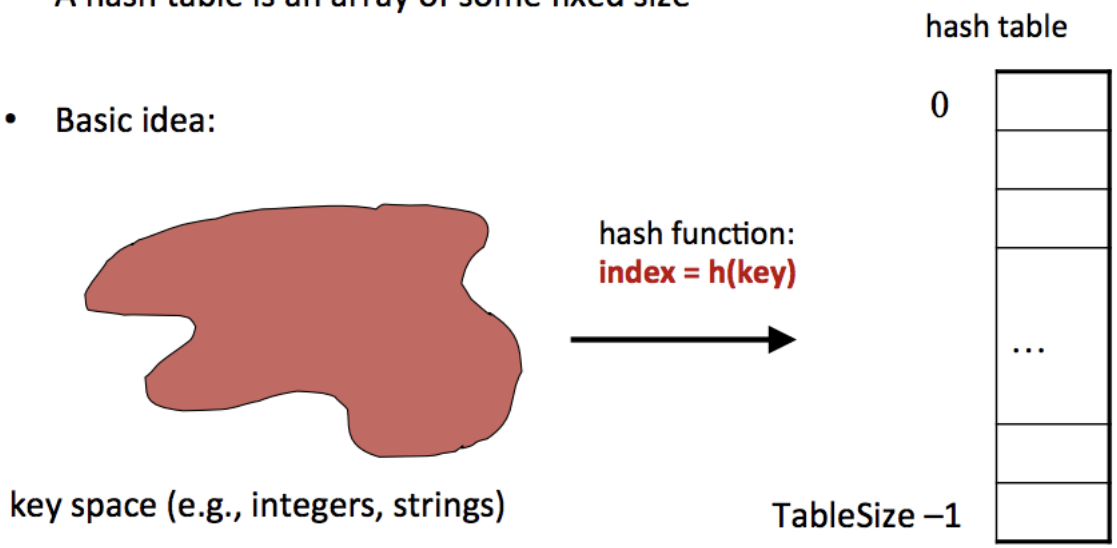


Example of a Search Tree

Handling Repeated States – Use efficient data structures

- **Use efficient data structures to keep the explored nodes.**
- Hash Tables
  - Insertion and look up in constant time.
- Duplicate checking
  - Canonical form, sorted list or other efficient methods.

- Aim for constant-time (i.e., $O(1)$) **`find`**, **`insert`**, and **`delete`**
  - "On average" under some often-reasonable assumptions
- A hash table is an array of some fixed size
- Basic idea:



Handling Repeated States – Check for some of the cases

- **Never return to the state you have just come from**
  - Prevent the node expansion function from generating any node successor that is the same state as the node's parent.
- **Never create search paths with cycles in them**
  - The node expansion function must be prevented from generating any node successor that is the same state as any of the node's ancestors
- **Practical techniques but sub-optimal**

Handling Repeated States – Remember all the visited nodes

- **Never generate states that have already been generated before.**
- Maintain an explored list (Graph search)
- Optimal approach
- Memory inefficient, why?
  - Exponential number of nodes in the tree
    - E.g., 8-puzzle problem, we have 9! = 362,880 states.
  - Duplicate checking of states also adds time.

function GRAPH-SEARCH(*problem*) **returns** a solution, or failure
  initialize the frontier using the initial state of *problem*
  initialize the explored set to be empty
  loop do
    if the frontier is empty **then return** failure
    choose a leaf node and remove it from the frontier
    if the node contains a goal state **then return** the corresponding solution
    add the node to the explored set
    expand the chosen node, adding the resulting nodes to the frontier
      only if not in the frontier or explored set

General Procedure: Best first search

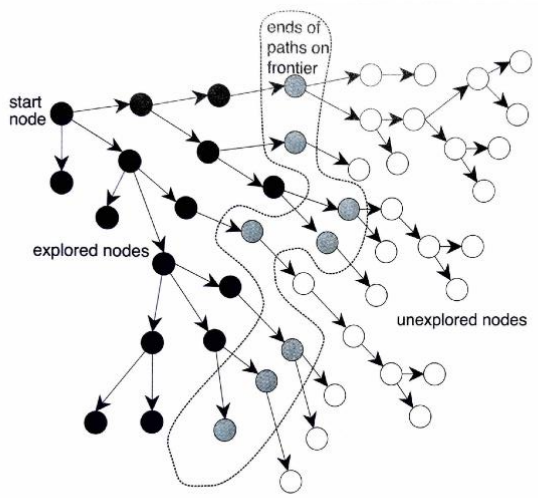
[role of frontiers and explored node lists, different prioritizations in the frontier]

Organize Search in a "Best First" Manner

Core technique:

1. Explore nodes from the starting state. Continue downstream.
2. Maintain nodes in a frontier.
3. When we obtain a node from the frontier, we see if this state satisfied the goal condition.

Remember: we are doing exploration without building the whole graph.




```
function BEST-FIRST-SEARCH(problem, f) returns a solution node or failure
  node ← NODE(STATE=problem.INITIAL)
  frontier ← a priority queue ordered by f, with node as an element
  reached ← a lookup table, with one entry with key problem.INITIAL and value node
  while not IS-EMPTY(frontier) do
    node ← POP(frontier)
    if problem.IS-GOAL(node.STATE) then return node
    for each child in EXPAND(problem, node) do
      s ← child.STATE
      if s is not in reached or child.PATH-COST < reached[s].PATH-COST then
        reached[s] ← child
        add child to frontier
  return failure
```

Best-First Search

Note:

1. Within the frontier need a strategy to prioritize which node to explore first.
2. There can be different ways to prioritize the partial solutions on the frontier.
   - Note we have only seen small part of the state space till then.
   - Leads to a rich set of algorithms.

```
function BEST-FIRST-SEARCH(problem, f) returns a solution node or failure
  node ← NODE(STATE=problem.INITIAL)
  frontier ← a priority queue ordered by f, with node as an element
  reached ← a lookup table, with one entry with key problem.INITIAL and value node
  while not IS-EMPTY(frontier) do
    node ← POP(frontier)
    if problem.IS-GOAL(node.STATE) then return node
    for each child in EXPAND(problem, node) do
      s ← child.STATE
      if s is not in reached or child.PATH-COST < reached[s].PATH-COST then
        reached[s] ← child
        add child to frontier
  return failure

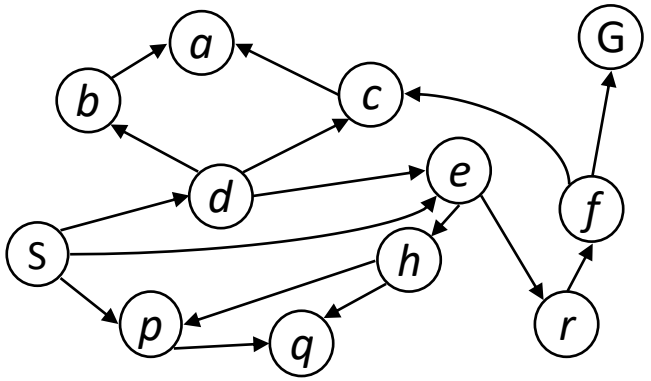
function EXPAND(problem, node) yields nodes
  s ← node.STATE
  for each action in problem.ACTIONS(s) do
    s' ← problem.RESULT(s, action)
    cost ← node.PATH-COST + problem.ACTION-COST(s, action, s')
    yield NODE(STATE=s', PARENT=node, ACTION=action, PATH-COST=cost)
```

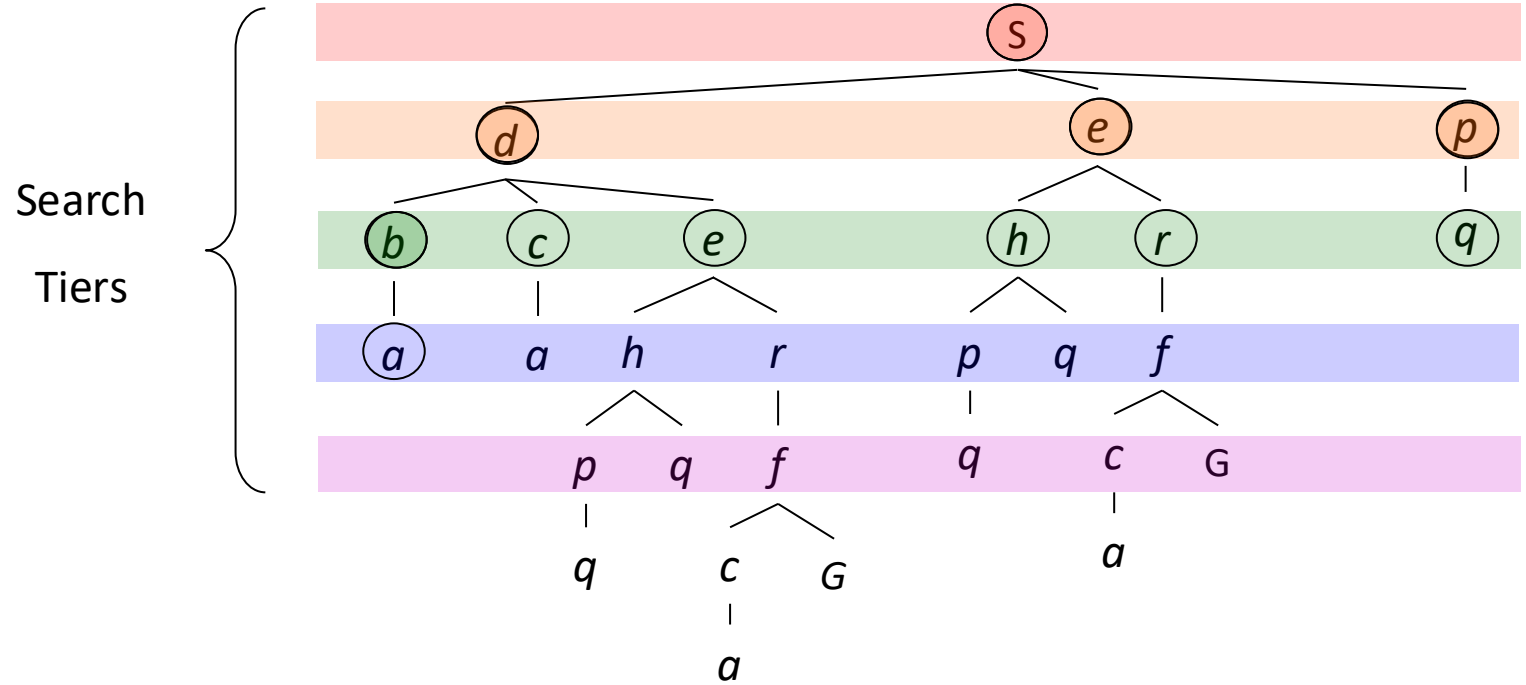
Breadth-First Search (BFS)

Strategy: expand a shallowest unexplored node first.

All the successors of a node are expanded, then their successors and so on.

Implementation: Frontier is a FIFO queue

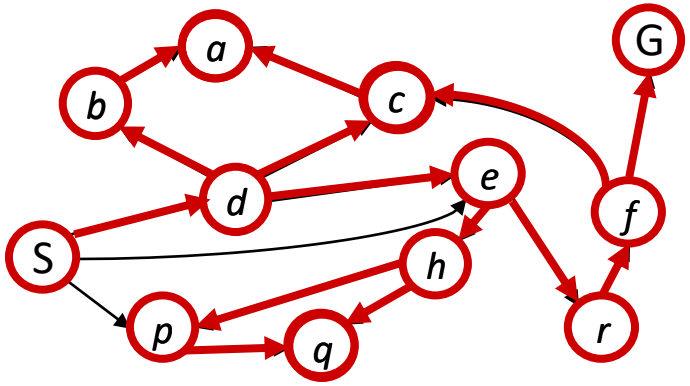


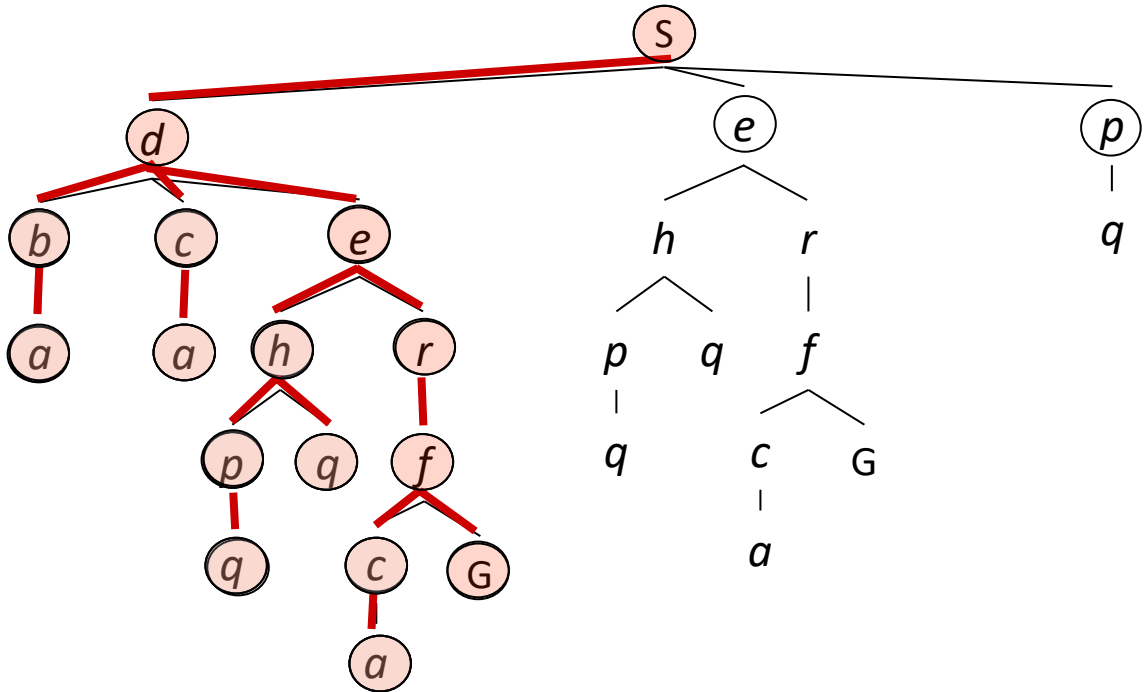


Depth-First Search (DFS)

Strategy: expand a deepest node first

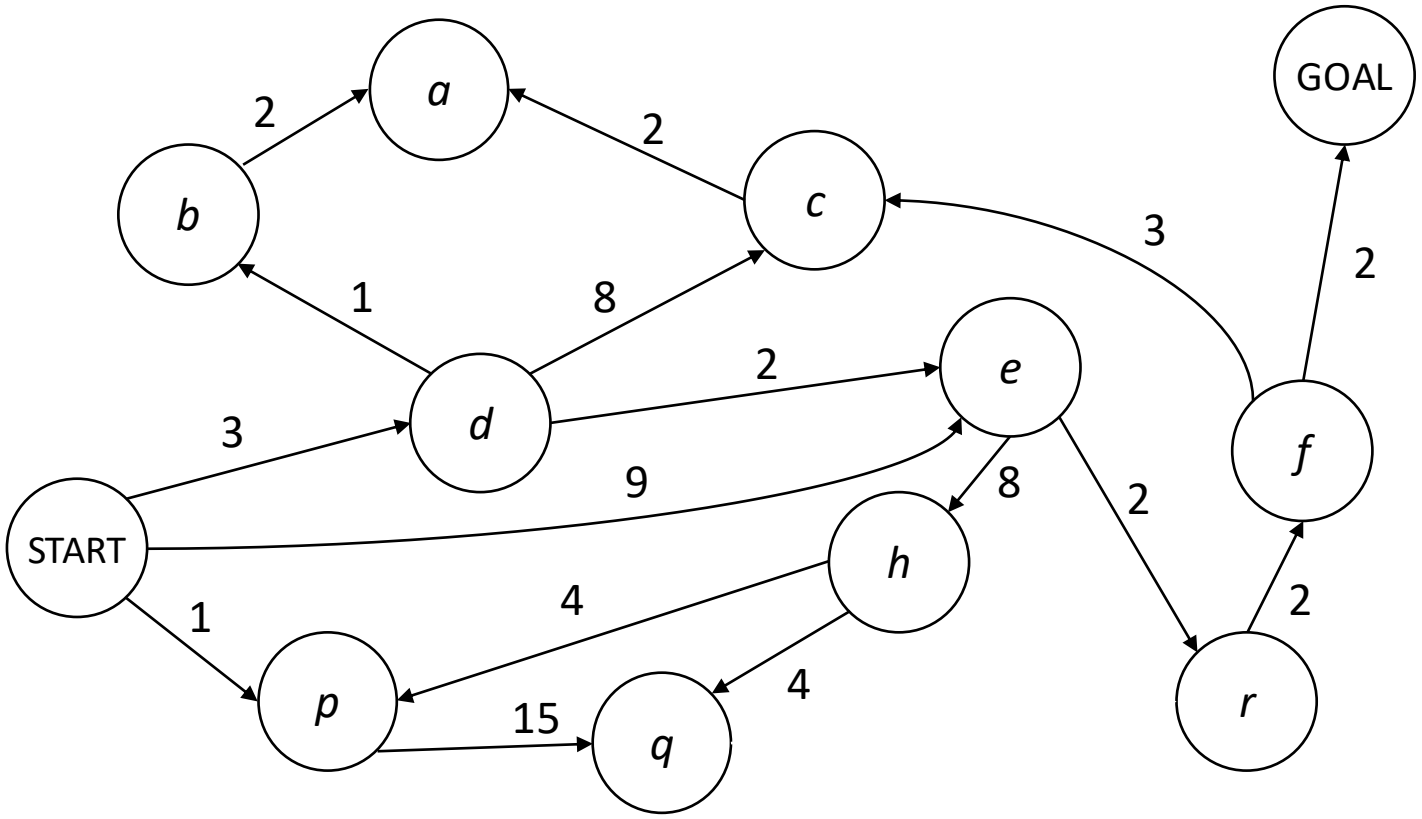
Implementation: Frontier is a LIFO stack





Guiding search by costs instead of depth

- Till now, the cost on the edges was not considered
  - Worked with solution depths.
- Solution was found in terms of number of actions.
  - Did not find the least-cost path.
  - BFS, DFS ….
- Next
  - Cost-sensitive search



UCS is also Djikstra's algorithm!
[Note this is from start to goal]

Uniform Cost Search (UCS)

- The first goal node generated may be on the sub-optimal path.
- If the new path is better than the old path, then discard the old one.

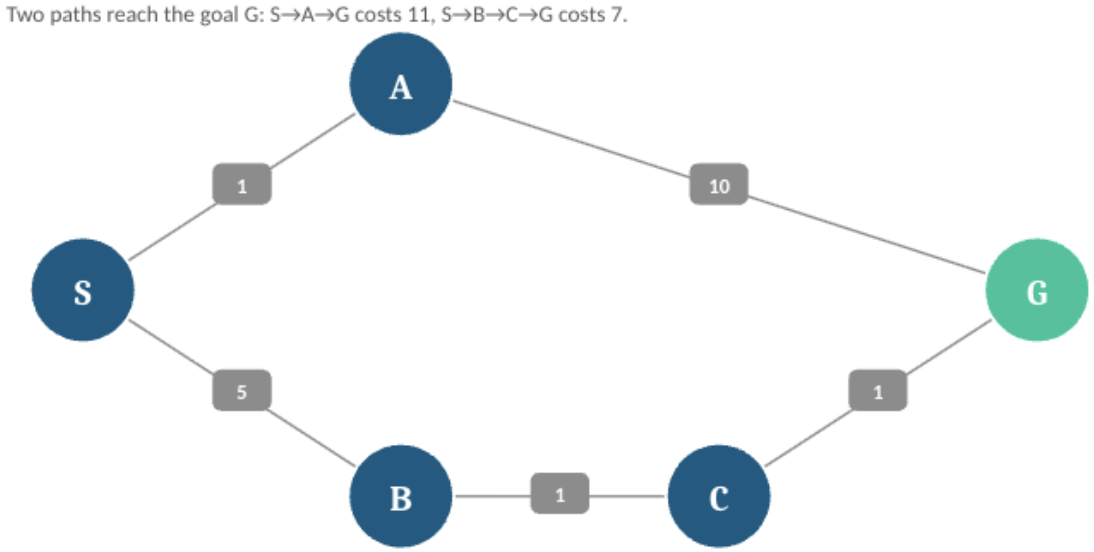
```
function UNIFORM-COST-SEARCH(problem) returns a solution, or failure
  node ← a node with STATE = problem.INITIAL-STATE, PATH-COST = 0
  frontier ← a priority queue ordered by PATH-COST, with node as the only element
  explored ← an empty set
  loop do
    if EMPTY?(frontier) then return failure
    node ← POP(frontier)  /* chooses the lowest-cost node in frontier */
    if problem.GOAL-TEST(node.STATE) then return SOLUTION(node)
    add node.STATE to explored
    for each action in problem.ACTIONS(node.STATE) do
      child ← CHILD-NODE(problem, node, action)
      if child.STATE is not in explored or frontier then
        frontier ← INSERT(child, frontier)
      else if child.STATE is in frontier with higher PATH-COST then
        replace that frontier node with child
```

Example

Role of the frontier in tracking potential solutions to the goal

- Start, S and the goal, G. Edge costs are in grey.
- The path to the goal is returned once the popped node satisfies the goal check function.
- The sub-optimal path is on the frontier earlier. The optimal path encountered later is expanded first.

Ensures that once the node is expanded and the goal is found, the solution is optimal.



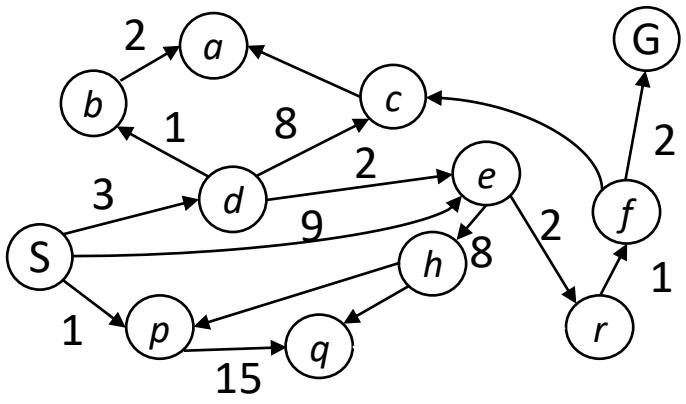
Iteration	Node popped (cost)	Expand into (cost)	Frontier after this step (node:cost)
0	—	—	{S:0}
1	S (0)	A (1), B (5)	{A:1, B:5}
2	A (1)	G via A → 1+10 = 11	{B:5, G:11}
3	B (5)	C → 5+1 = 6	{C:6, G:11}
4	**C (6)**	G via C → 6+1 = 7, solution cost is lower than the existing G:11, so update it	{G:7} Note: old G:11 entry is discarded/superseded
5	**G (7)**	Goal is reached. **[A-B-C-G]**	{NIL}

Declaration: Graph rendering was aided by AI.

Uniform Cost Search (UCS)

Strategy: expand a "cheapest" cost node first:

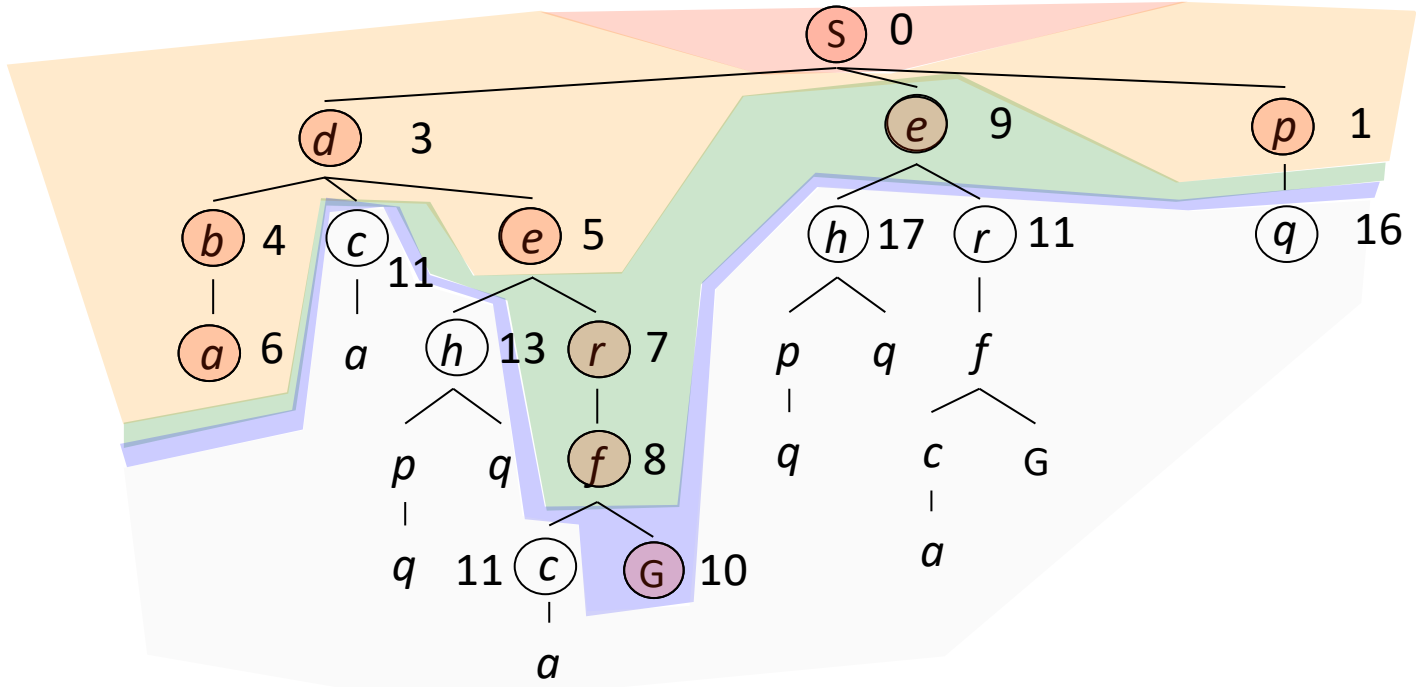
Frontier is a Priority Queue (Priority: cumulative cost so far)



Intuition for the f function in UCS

The partial solutions that are low-cost are likely to lead to optimal (lowest) cost paths in future. Hence, prioritize them.

Cost contours during search



Different Mechanisms for Search

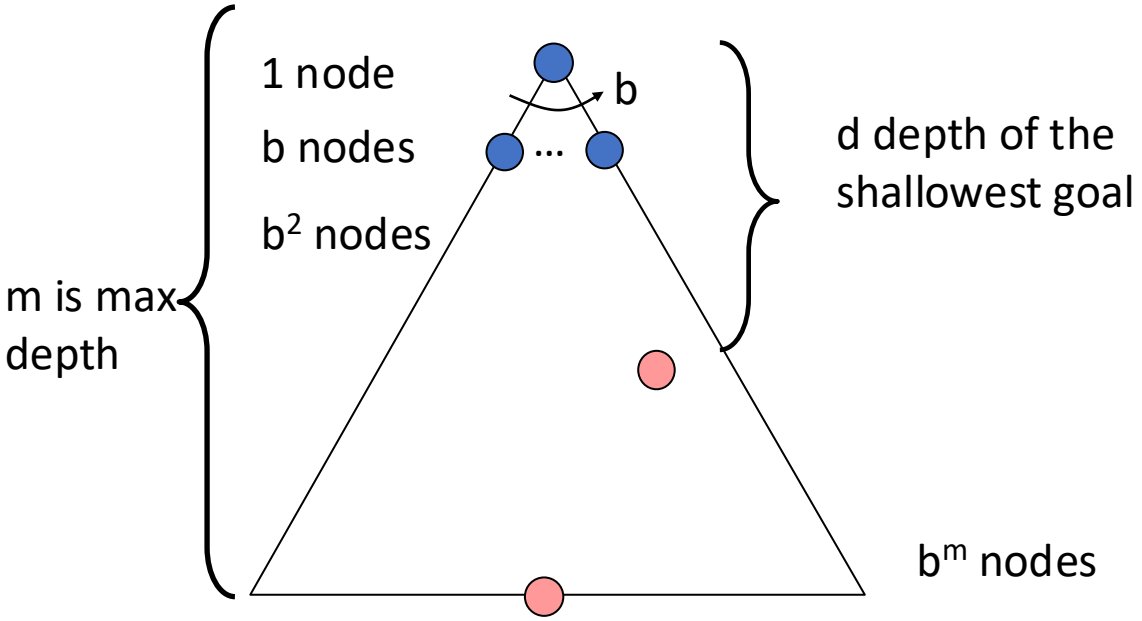
[Varied prioritizations in the frontier, Properties]

Properties of Search Algorithms

- Time Complexity
  - The **number of nodes** generated during search.
- Space Complexity
  - The **maximum number of nodes stored in memory** during search.
- Optimality
  - Is the plan returned by the search algorithm the **optimal** ?
- Completeness
  - Is the search algorithm guaranteed to **find a solution** when there is one?
  - Should not happen that there is a solution but the algorithm does not find it (e.g., infinite loop in a part of the state space)

Analysing problem-solving performance

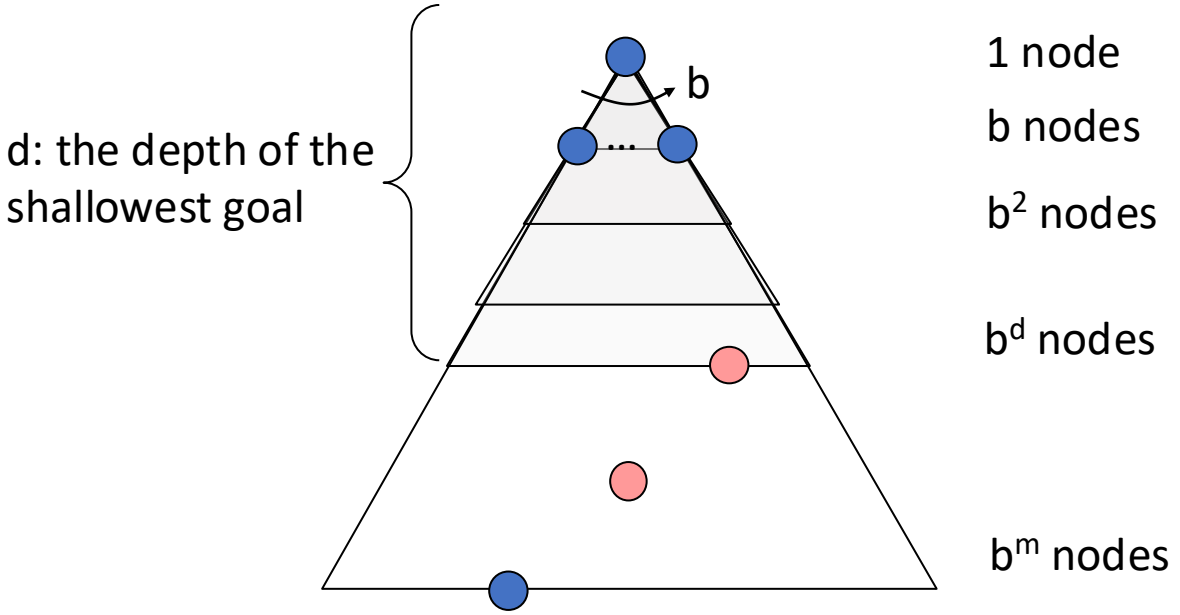
- **Diagram to model a search tree:**
  - b is the branching factor
  - m is the maximum depth
  - solutions at various depths
  - d is the depth of the shallowest goal node
- **Number of nodes in entire tree?**
  - $1 + b + b^2 + \ldots. b^m = \mathbf{O(b^m)}$
  - Each node can generate the b new nodes



Remember: there may be more than one goal states. Hence, we want to find the shallowest goal (optimal) solution.

Breadth First Search (BFS) Properties

- **Expansion Strategy**
  - Expands the **shallowest** unexplored node in the frontier of the search tree.
- **Time Complexity**
  - Search takes time **$O(b^d)$**
- **Space Complexity**
  - Frontier nodes **$O(b^d)$**
  - Explored nodes **$O(b^{d-1})$**
  - *Memory requirement is a problem in BFS search.*



Animation courtesy Dan Klein, Peter Abbeel

Breadth First Search (BFS) Properties

Empirical analysis: Time and memory requirements for BFS.

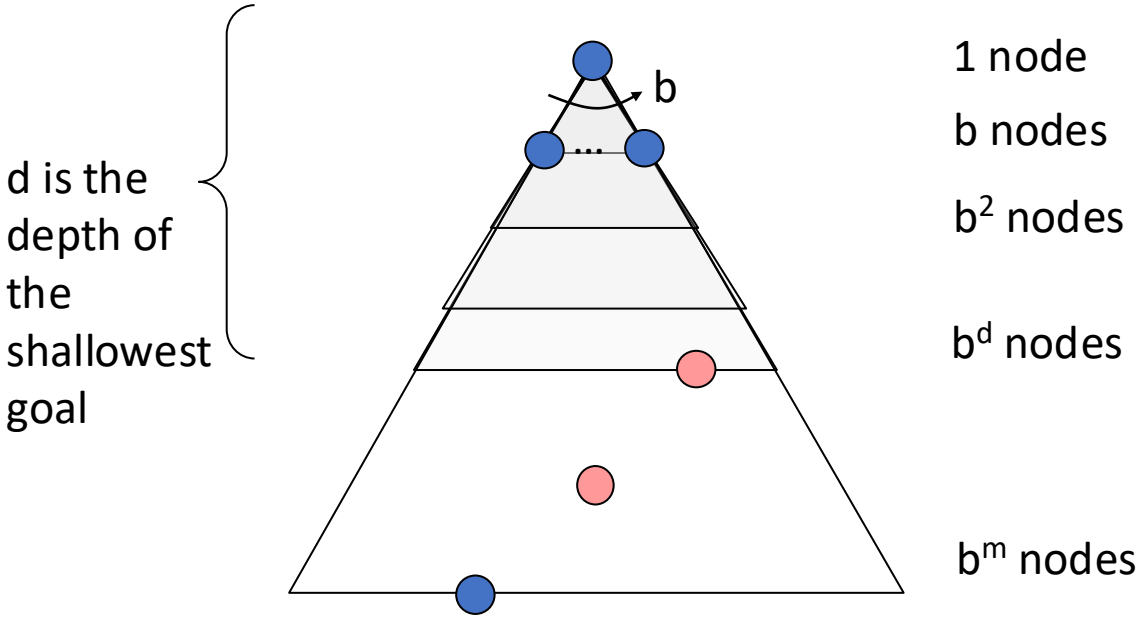
- Branching factor b = 10.
- One million nodes per second processed and 100 bytes per node used to store.

Depth	Nodes	Time	Memory
0	1	1 millisecond	100 bytes
2	111	.1 seconds	11 kilobytes
4	11,111	11 seconds	1 megabyte
6	10^6	18 minutes	111 megabytes
8	10^8	31 hours	11 gigabytes
10	10^{10}	128 days	1 terabyte
12	10^{12}	35 years	111 terabytes
14	10^{14}	3500 years	11,111 terabytes

Takeaway: Memory requirement is a **"big"** problem for BFS! Note that BFS (unlike DFS) has the advantage of returning the path to the shallowest goal first.

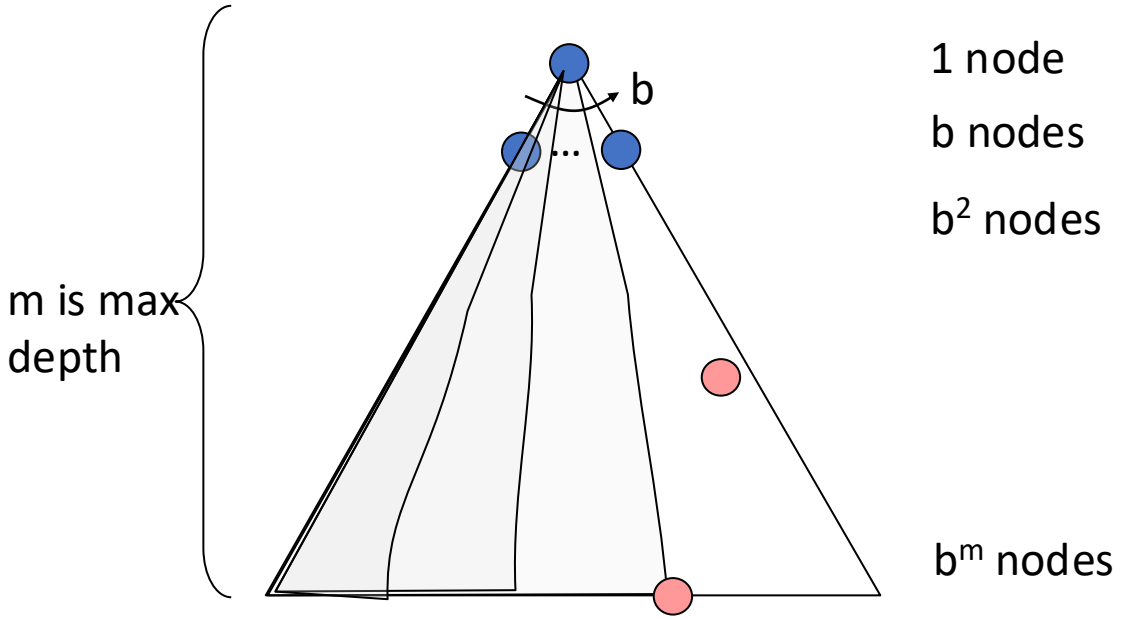
Breadth First Search (BFS) Properties

- **Is it complete?**
  - **Yes.**
  - The shallowest goal is at a finite depth, d
  - If the branching factor, b, is finite then BFS will find it.
- **Is it optimal?**
  - **Yes.** If the path cost is a non-decreasing function of depth.
    - For example, if all edge costs are equal.



Depth First Search (DFS) Properties

- **Expansion Strategy**
  - Expands the **deepest** unexplored node in the frontier of the search tree
- **Time Complexity**
  - Worst case: processes the whole tree.
  - If m is finite, takes time **$O(b^m)$**
- **Space Complexity**
  - Frontier stores:
    - Single path from the root to the leaf node.
    - Sibling nodes on the path that are unexplored.
  - **Memory requirement is low O(bm)**
- **Is it complete?**
  - **Yes**, if m is finite. Eventually, finds the path.
- **Is it optimal?**
  - **No**, it finds the "leftmost" solution, regardless of depth or cost



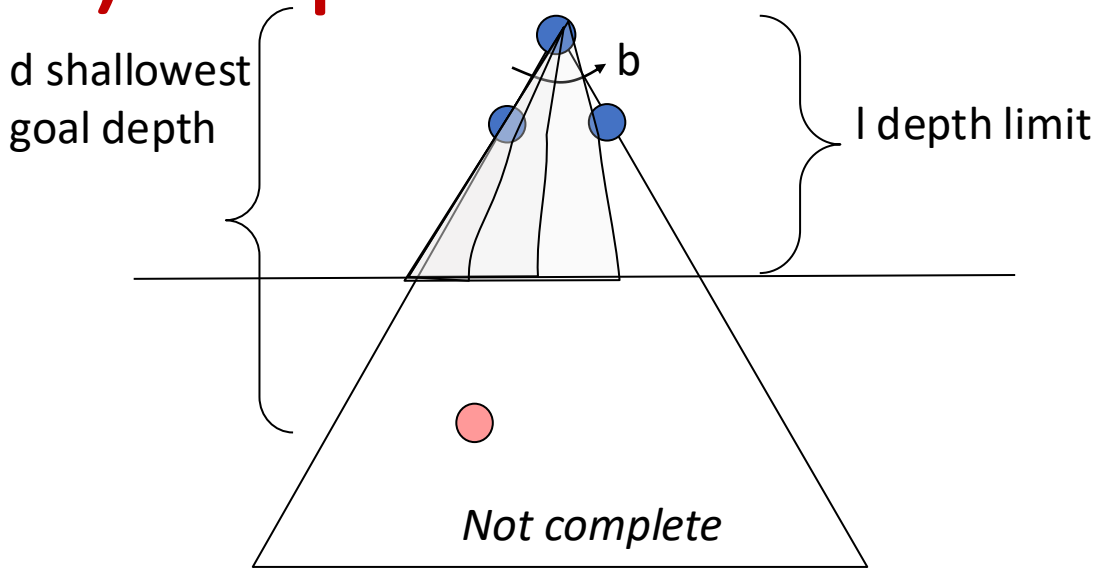
Note: Variant of graph search where the goal test is applied at node generation time. Space saving.

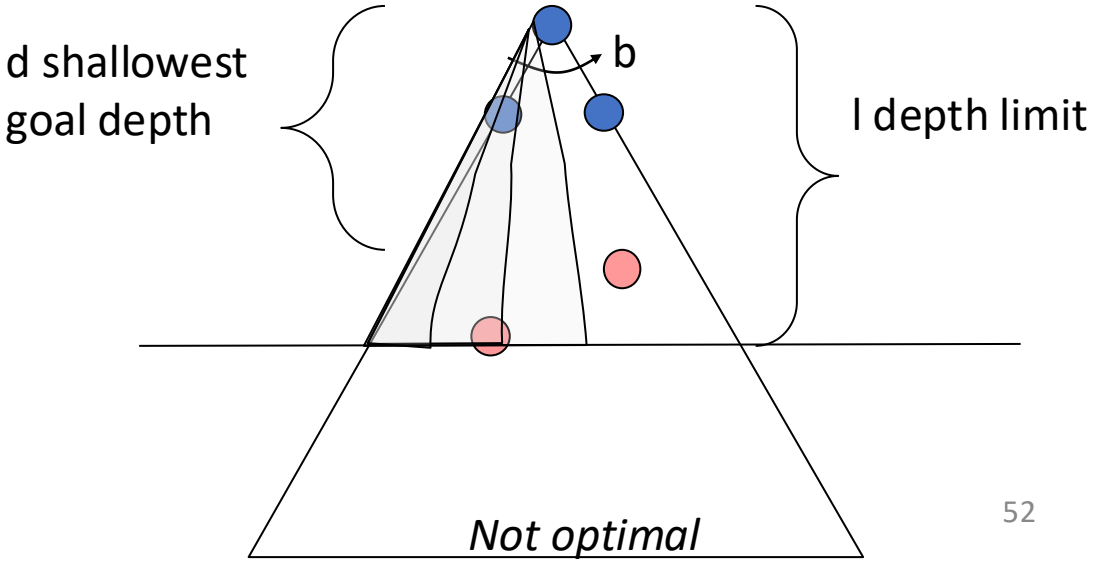
What if we only search till a certain depth?

- **Observe**
  - Depth First Search fails when the maximum goal depth is not known ahead of time for a domain.
  - Bound the maximum depth till when the search is performed.
- **Depth Limited Search (DLS)**
  - Perform **Depth First Search** but **terminate** a path either if a **goal state** is found or if the **maximum allowed depth** is reached.
  - Restrict the depth of search (supply a depth limit, l). Equivalent to assuming that nodes at level "l" have no successors.
  - In essence, force DFS to back track after a limit (in essence making DFS move breadth wise at the limit).

Depth Limited Search (DLS) Properties

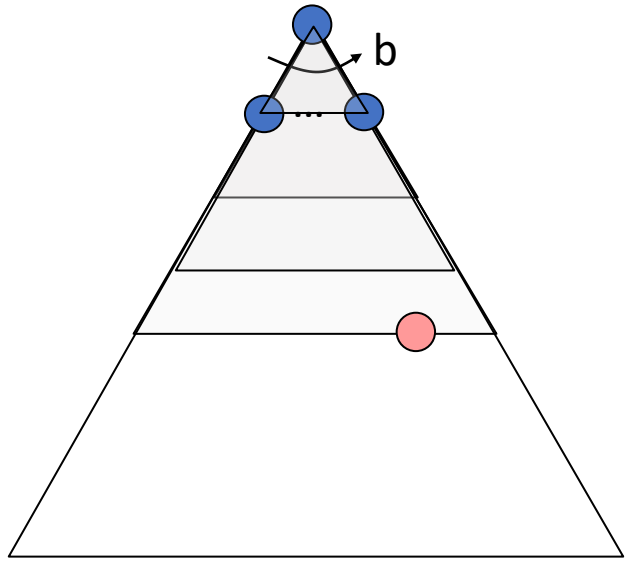
- **Termination**
  - Always terminates.
- **Time Complexity**
  - Worst case: processes the whole tree till l.
  - Time **$O(b^l)$**
- **Space Complexity**
  - Frontier is managed like Depth First Search.
  - Memory **O(bl).**
- **Is it complete?**
  - **Not complete** when goal depth is greater than the limit (d>l)
- **Is it optimal?**
  - **Not optimal** when the limit is greater than the goal depth (l > d)





Iterative Deepening Search

- Combine DFS's space advantage with BFS's shallow-solution advantages
  - Run a DLS with depth limit 1. If no solution…
  - Run a DLS with depth limit 2. If no solution…
  - Run a DLS with depth limit 3. …..




```
function ITERATIVE-DEEPENING-SEARCH(problem) returns a solution, or failure
    for depth = 0 to ∞ do
        result ← DEPTH-LIMITED-SEARCH(problem, depth)
        if result ≠ cutoff then return result
```

Note the arrow indicating the node explored.

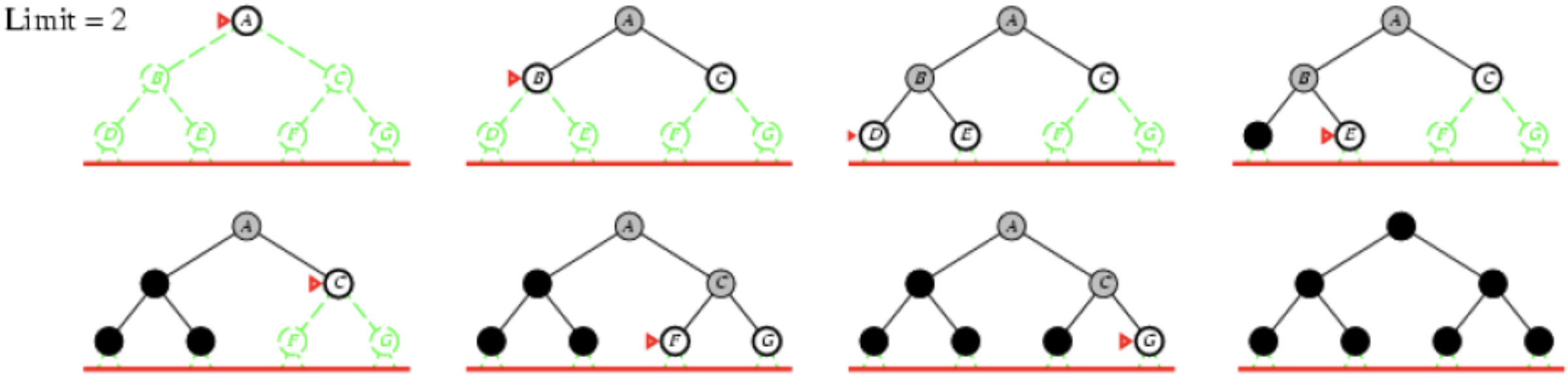
Iterative Deepening: Example

Limit = 1



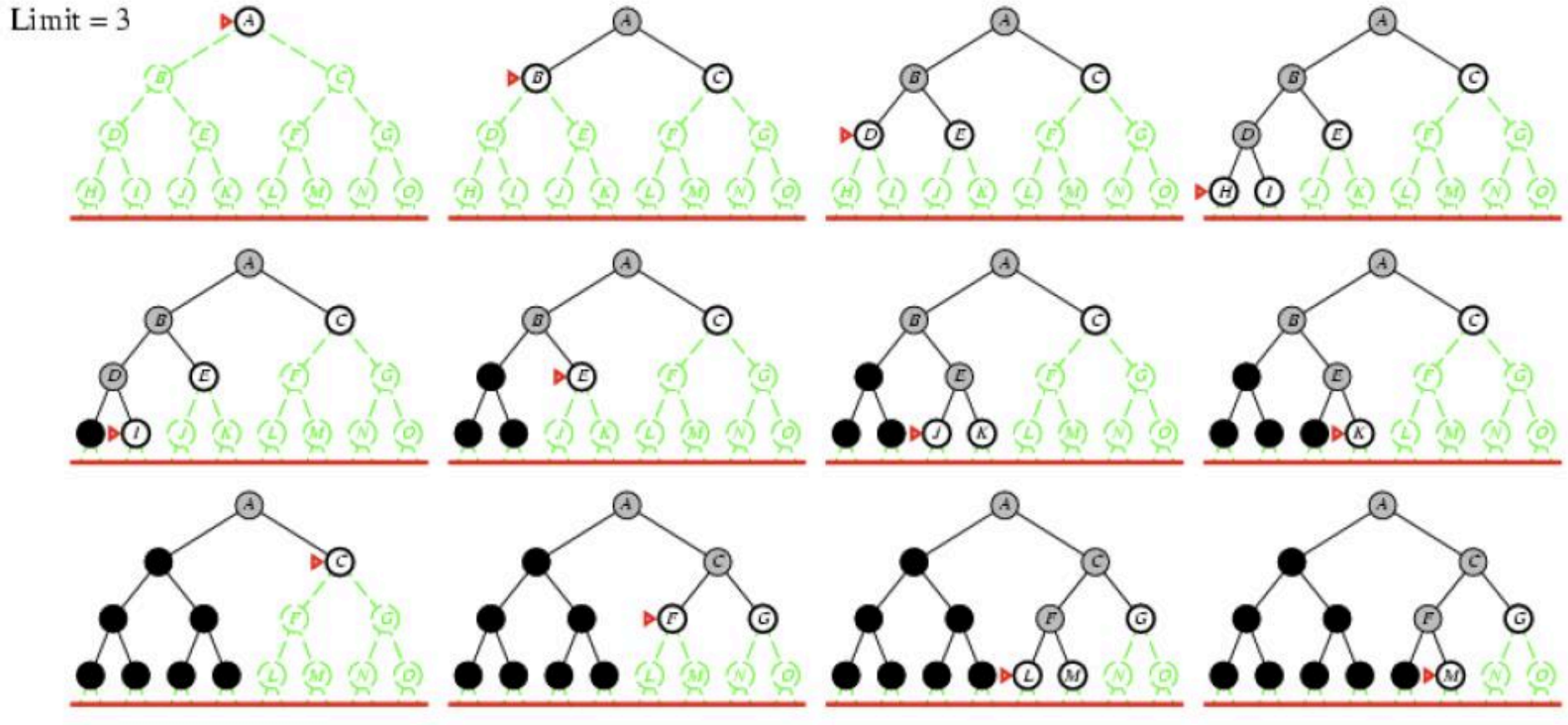
Note the arrow indicating the node explored.

Iterative Deepening: Example



Note the arrow indicating the node explored.

Iterative Deepening: Example



Think: what does such iterative depth limit search buy us?

Iterative Deepening: Pseudo-code

Core Idea

- During Search IDS uses DFS, hence memory usage is less.
- The depth causes the DFS search to backtrack and move breadth wise.
- The iterative part ensures that the shallowest goal is found first.

```
function ITERATIVE-DEEPENING-SEARCH(problem) returns a solution node or failure
  for depth = 0 to ∞ do
    result ← DEPTH-LIMITED-SEARCH(problem, depth)
    if result ≠ cutoff then return result

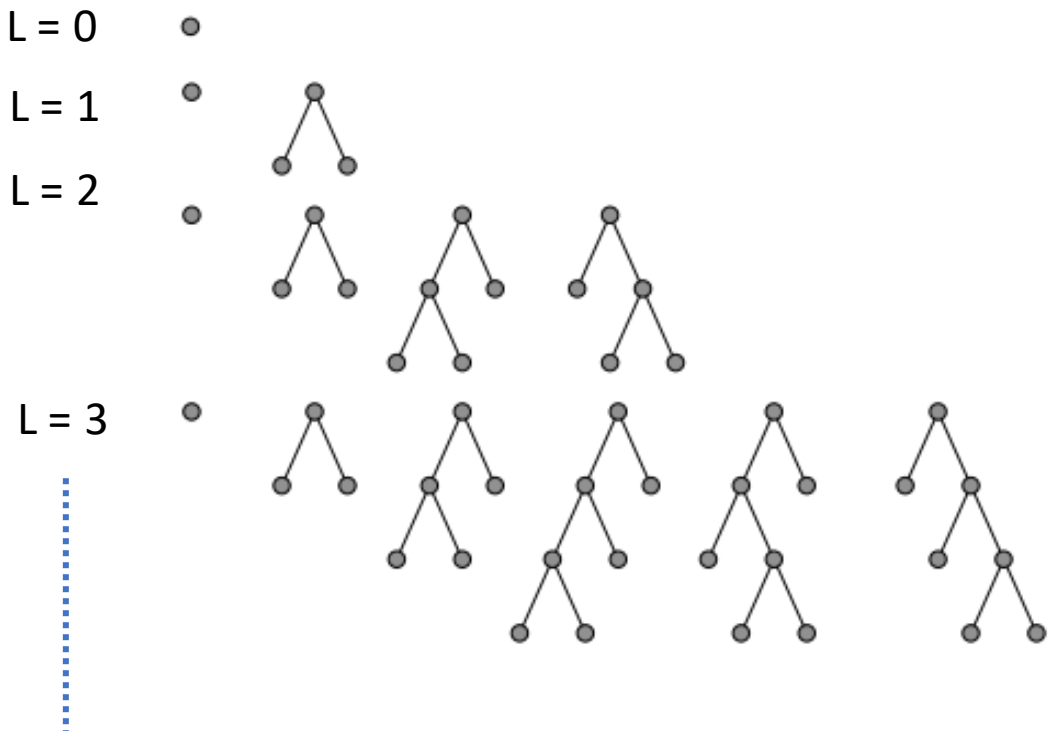
function DEPTH-LIMITED-SEARCH(problem, ℓ) returns a node or failure or cutoff
  frontier ← a LIFO queue (stack) with NODE(problem.INITIAL) as an element
  result ← failure
  while not IS-EMPTY(frontier) do
    node ← POP(frontier)
    if problem.IS-GOAL(node.STATE) then return node
    if DEPTH(node) > ℓ then
      result ← cutoff
    else if not IS-CYCLE(node) do
      for each child in EXPAND(problem, node) do
        add child to frontier
  return result
```

Figure 3.12 Iterative deepening and depth-limited tree-like search. Iterative deepening repeatedly applies depth-limited search with increasing limits. It returns one of three different types of values: either a solution node; or *failure*, when it has exhausted all nodes and proved there is no solution at any depth; or *cutoff*, to mean there might be a solution at a deeper depth than ℓ. This is a tree-like search algorithm that does not keep track of *reached* states, and thus uses much less memory than best-first search, but runs the risk of visiting the same state multiple times on different paths. Also, if the IS-CYCLE check does not check *all* cycles, then the algorithm may get caught in a loop.

Pseudo-code from AIMA 4th edition. Pseduo-code is online.

Iterative Deepening Properties

- **Time Complexity**
  - Time **O(b^d)**
- **Space Complexity**
  - Memory **O(bd)**
  - Linear memory requirement like **DFS**
- **Is it complete?**
  - **Yes.** Complete like **BFS**
- **Is it optimal?**
  - **Yes.** Optimal like BFS (if costs are non-decreasing function of path length)
- **Relevance**
  - Preferred method for large state spaces where maximum depth of a solution is unknown



Assessing the impact of repeated node generation

- Is it wasteful to generate nodes again and again?
  - Not really! If one examines the order of complexity.
  - The lowest level contributes the maximum. Overhead of re-generation is not significant in practice.
- Asymptotic time complexity is same as BFS: **$O(b^d)$**

- Number of nodes generated in a depth-limited search to depth *d* with branching factor *b*:
  - $N_{DLS} = b^0 + b^1 + b^2 + \ldots + b^{d-2} + b^{d-1} + b^d$
- Number of nodes generated in an iterative deepening search to depth *d* with branching factor *b*:
  - $N_{IDS} = (d+1)b^0 + d\ b^{\wedge 1} + (d-1)b^{\wedge 2} + \ldots + 3b^{d-2} + 2b^{d-1} + 1b^d$
- Asymptotic ratio: (b+1)/(b-1)
- For *b = 10*, *d = 5*,
  - $N_{DLS} = 1 + 10 + 100 + 1{,}000 + 10{,}000 + 100{,}000 = 111{,}111$
  - $N_{IDS} = 6 + 50 + 400 + 3{,}000 + 20{,}000 + 100{,}000 = 123{,}456$
- Overhead = (123,456 - 111,111)/111,111 = 11%

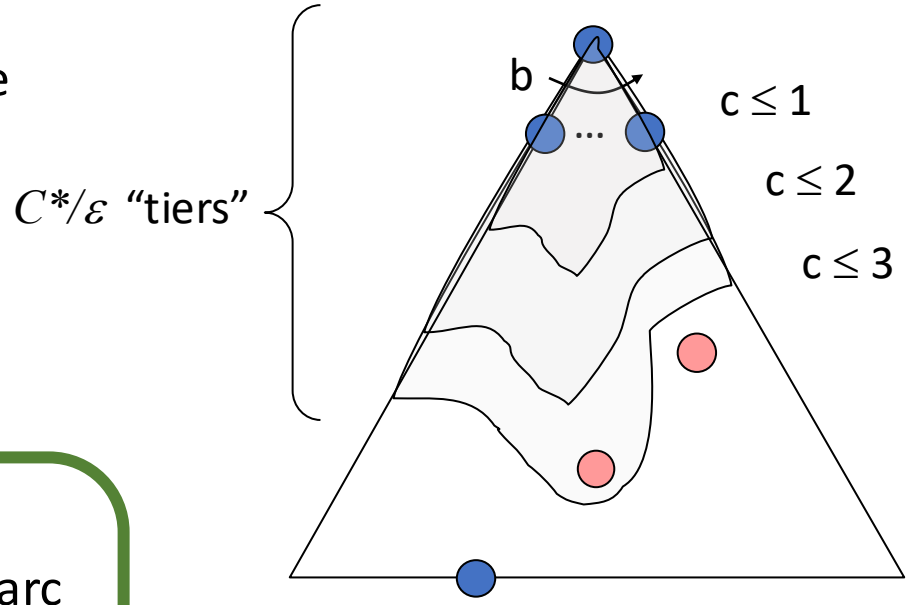
No. of times generated.

The overhead is not too bad!

Adapted from Prof. Mausam's slide

Uniform Cost Search (UCS) Properties

- What nodes does UCS expand?
  - Guided by costs and not the depth.
  - If that solution costs C^* and action cost at least ε, then the "effective depth" is roughly C^*/ε
  - Takes time **O($b^{C^*/\varepsilon}$)** (exponential in effective depth)
- How much space does the frontier take?
  - **O($b^{C^*/\varepsilon}$)**



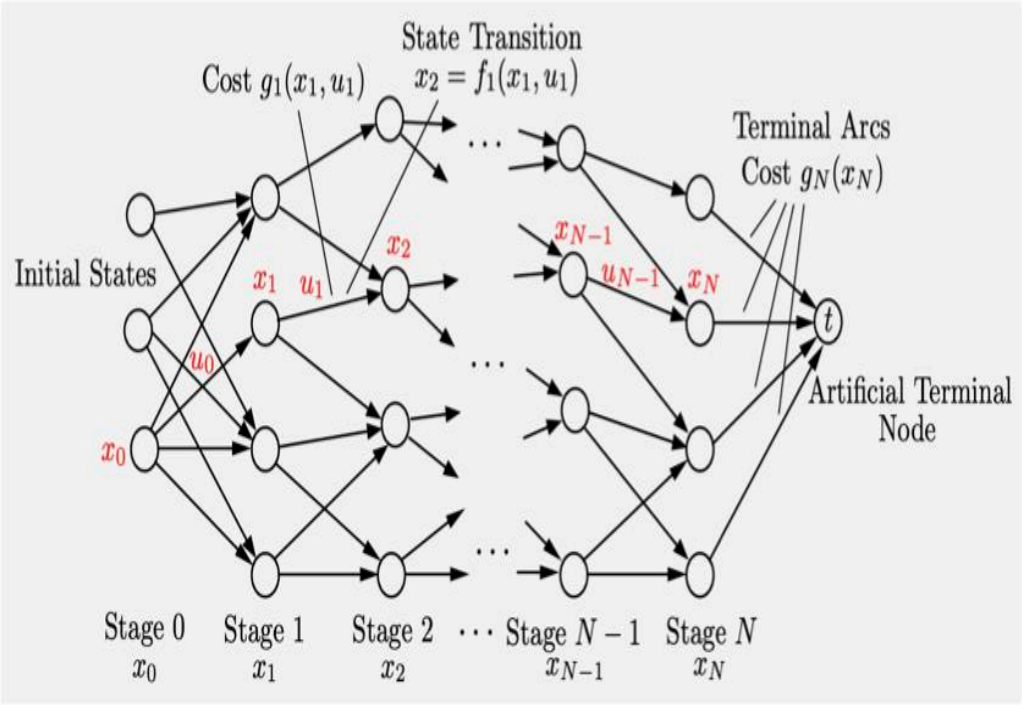
- Is it complete?
  - **Yes.** Assuming best solution has a finite cost and minimum arc cost is positive, yes!
- Is it optimal?
  - **Yes.** (Proof via contradiction)

What if we bound the frontier size?

In reality – our memory is always bounded, we will run out at some time.

- **Idea: Keep a *maximum size* of the frontier.**
  - Only keep the *k* best candidates for expansion, discard the rest.
- Advantage:
  - More space efficient
- Disadvantage
  - May throw away a node that is on the solution path

- Complete? **No.**
- Optimal? **No.**
- Very popular in practice

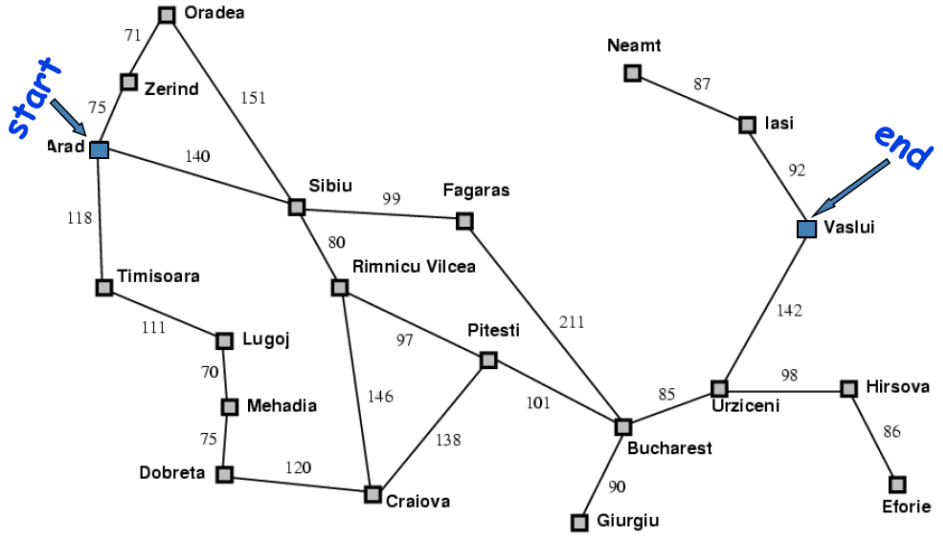


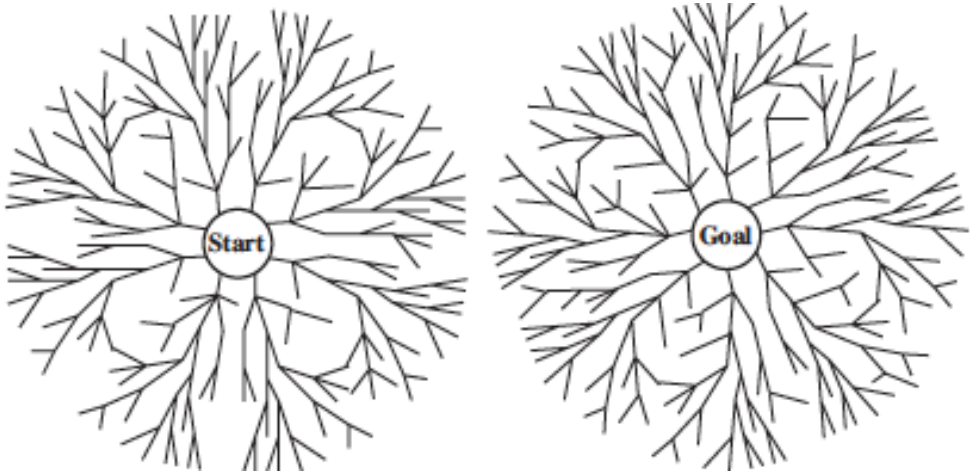
The procedure is called Beam Search.

What if we search from both ends?

Bi-directional Search

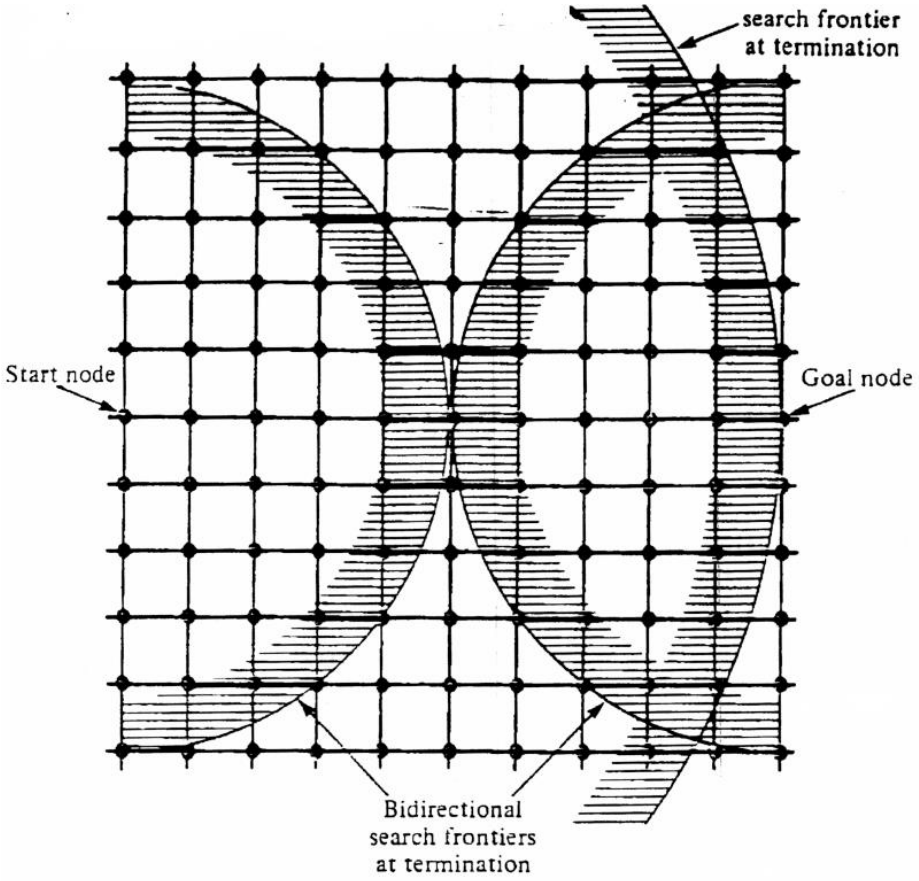
- Run one search forward from the initial state.
- Run another search backward from the goal.
- Stop when the two searches meet in the middle.





Bi-directional Search

- Space and time complexity
  - **$O(b^{d/2})$**
  - **$b^{d/2} + b^{d/2}$ is smaller than b^d**
  - $10^8+10^8 = 2.10^8 << 10^{16}$
- Needs an efficiently computable *Predecessor()* function
  - Difficult: e.g., predecessors of checkmate in chess?
- What if there are several goal states?
  - Create a new dummy goal state whose predecessors are the actual goal states.



Summary Table

Criterion	Breadth-First	Uniform-Cost	Depth-First	Depth-Limited	Iterative Deepening
Complete?	Yes	Yes	No	No	Yes
Time	$O(b^{d+1})$	$O(b^{\lceil C^*/\epsilon \rceil})$	$O(b^m)$	$O(b^l)$	$O(b^d)$
Space	$O(b^{d+1})$	$O(b^{\lceil C^*/\epsilon \rceil})$	$O(bm)$	$O(bl)$	$O(bd)$
Optimal?	Yes	Yes	No	No	Yes

- The comparison is for the tree-search version of the algorithms.
- For graph searches, DFS is complete for finite state spaces and that the space time complexities are bounded by the size of the state space.
- Source: AIMA

Perspective:

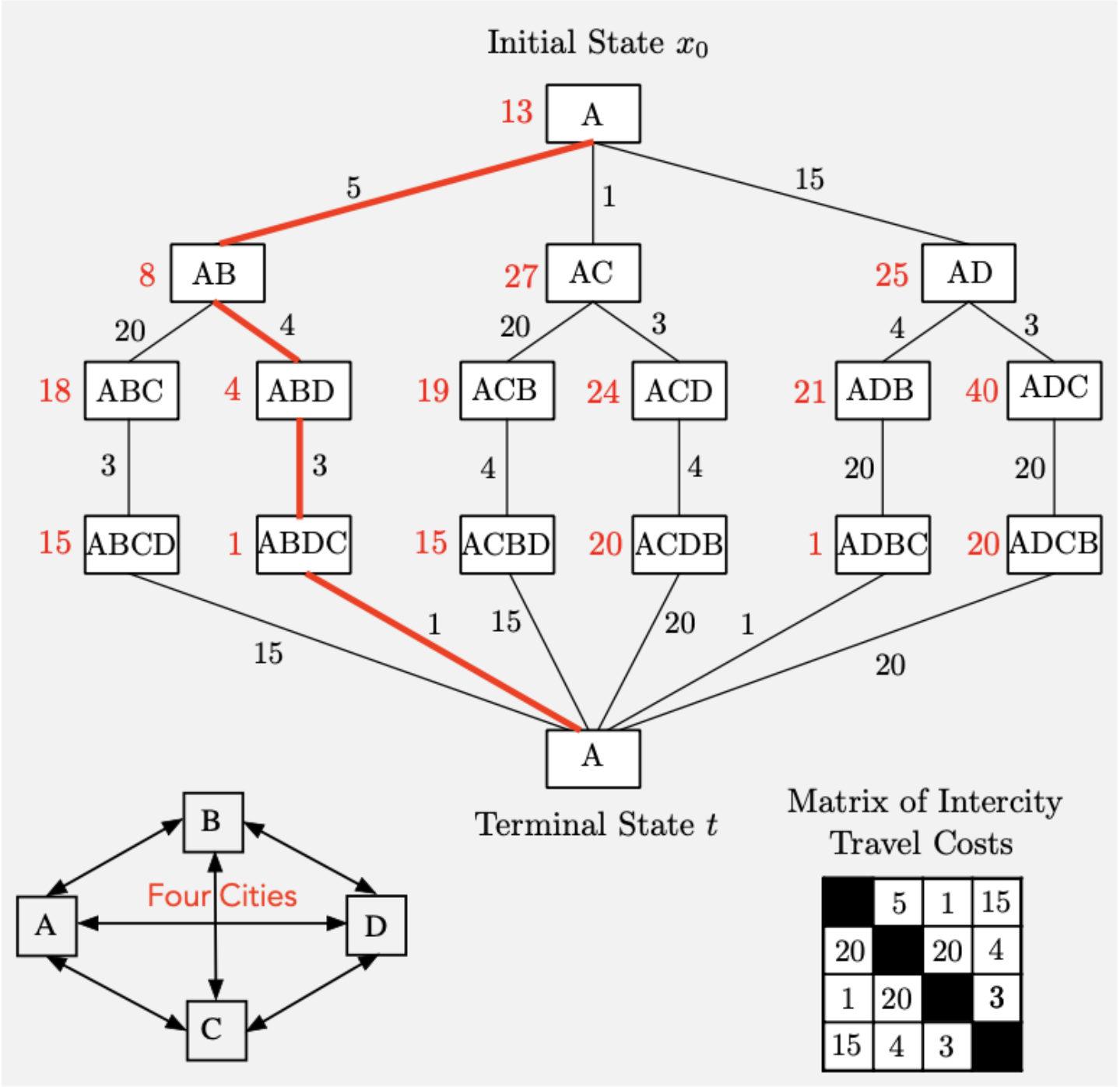
Search as a tool to solve computationally hard problems

Solving the Travelling Salesman Problem

Goal find a tour that minimizes the costs.

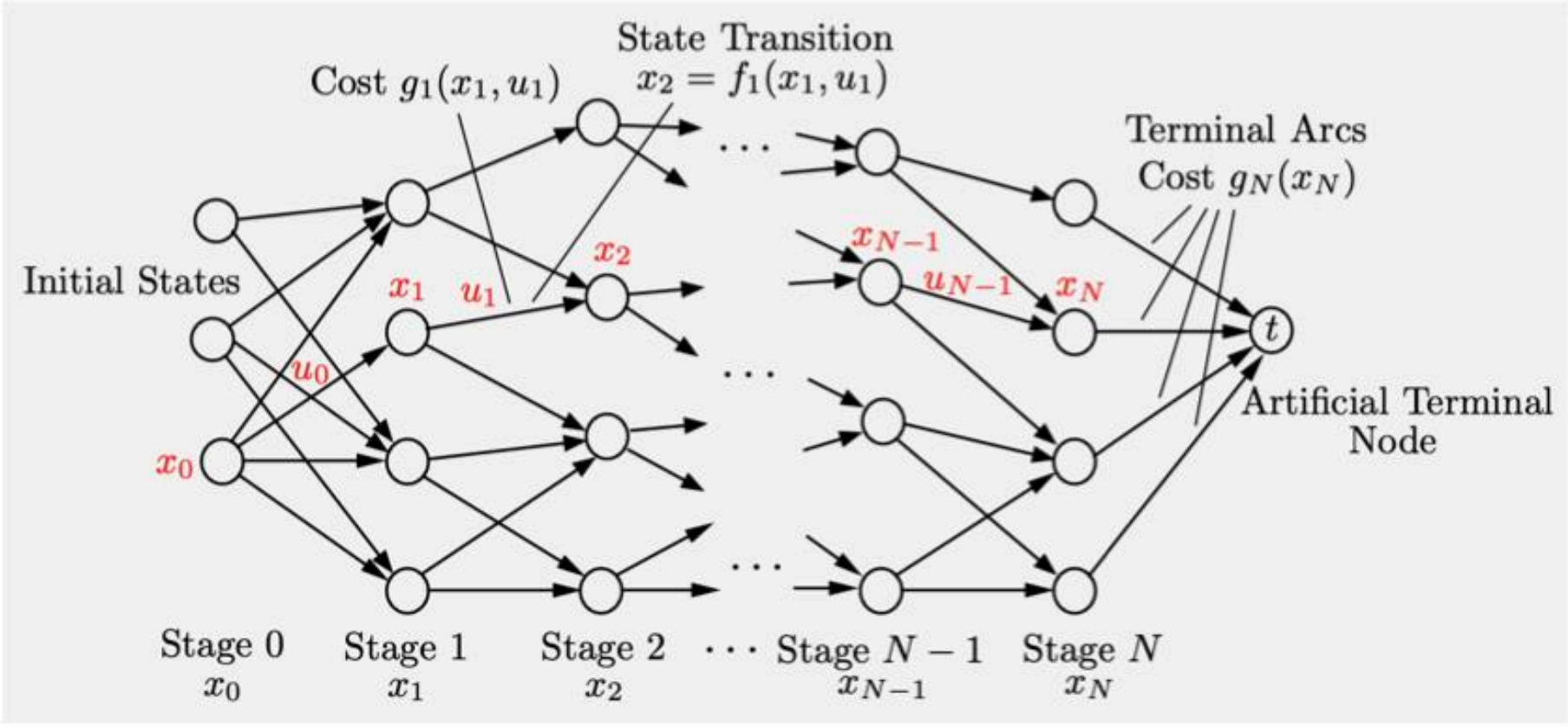
NP hard problem.

See right for the view as search on a state space graph.



A General View

We are looking for an optimal sequence of decisions (or controls, or actions) from the start to the goal.
After taking an action on a state we transition to a state and incur a cost.
Challenge is the combinatorial explosion of possibilities when we consider sequences.



Search algorithms provide an organized way to manage possibilities finally resulting in the solution.

Summary

- Algorithms that are given no information about the problem other than its definition.
  - No additional information about the state beyond that provided in the problem definition.
  - Generate successors and distinguish goal from non-goal states.
  - Hence, all we can do is move systematically between states until we stumble on a goal
  - Search methods are distinguished by the order in which the nodes are expanded.